

Heejong Park and Zoran Salcic

University of Auckland, New Zealand

Designing Software Systems Using SystemJ

Programming in SystemJ

January 2019

This document gives an introduction into SystemJ programming/design language and should be considered as work in progress. At this stage it is published only for internal use within the courses and research related to Embedded Systems Research Group at University of Auckland. All comments and suggestions by the readers are welcome and the authors will highly appreciate to get them directly. Please forward them directly to Zoran Salcic (z.salcic@auckland.ac.nz) and/or Heejong Park (Heejong.Park@auckland.ac.nz).

© All rights reserved by Heejong Park and Zoran Salcic. The distribution of the document for educational and non-commercial purposes is free.

Table of Content

Table of Content	i
Chapter 1 Introduction to SystemJ	1
1.1 Software Systems	1
1.1.1 Spatial and temporal properties	1
1.1.2 Reactivity.....	2
1.1.3 Concurrency and hierarchy	2
1.2 SystemJ Approach to Designing Software Systems.....	3
Chapter 2 SystemJ Language – Semantics, Syntax and Programming	5
2.1 Design Entities.....	5
2.2 Tick-Based Execution.....	7
2.3 SystemJ Kernel Statements.....	8
2.4 SystemJ Derived Statements	9
2.5 Writing Simple SystemJ Programs – Examples	11
2.5.1 Synchronous programs	11
2.5.2 GALS programs	14
2.6 SystemJ Helper Constructs	15
Chapter 3 Designing Software Systems	18
3.1 Designing Synchronous Systems	18
3.1.1 A Sequential Reactive Program	18
3.1.2 Designing Synchronous Systems – An Industrial Automation Example	22
3.2 Designing Asynchronous Systems	25
3.3 Full Synchronous System Example - ABRO	26
3.4 Full Asynchronous System Example - Producer Consumer ABRO (PCABRO)	28
Chapter 4 SystemJ Execution Models and Their Implementation	31
4.1 SystemJ Execution Models.....	31
4.1.1 Asynchronous Graph Code - AGRC	31
4.1.2 Execution Model for JVM platforms	32
4.1.3 Execution Model for Specialised Time-Predictable Platforms	34
4.2 SystemJ Execution Platforms	34
4.2.1 Java Virtual Machine (JVM) Based Platforms.....	35

4.2.2	Java Optimized Processor (JOP) and the Multi-Core Platforms	35
4.3	Developing SystemJ Programs	35
4.3.1	Program Components and Organisation	36
4.3.2	Specifying a Configuration File	38
4.4	Runtime System and its Customisation	41
4.4.1	InputFileReader extending GenericSignalReceiver	44
4.4.2	OutputFileWriter extending GenericSignalSender	45
Chapter 5	Distribution and Deployment of Software Systems	47
5.1	SystemJ Program Development and Deployment Strategies	47
5.2	Asynchronous Graph Code (AGRC)	48
5.3	Compiling and Running SystemJ Programs on Development Computers	50
5.4	Development Tools	51
5.4.1	The SystemJ Environment Simulator	52
5.4.2	The sysjnetapi Library	52
5.4.3	Pre-processing Local Configuration File	53
5.4.4	Signal interfaces	53
5.4.5	Gradle project	55
5.5	Experimental Features	56
Chapter 6	Comprehensive Case Studies	58
6.1	Frequency Relay	58
6.2	Industrial Automation and Robotics Collaborative System	65
6.2.1	Lid loader CD	68
6.2.2	Rotary table CD	70
6.2.1	Capper CD	72
6.2.2	Conveyor CD	73
6.2.3	Filler CD	74
Chapter 7	Conclusion	77
Appendix 1	Lists of the Elements and Attributes of the Configuration File	79
Appendix 2	SystemJ Tools and Example Programs	81
References		82

Chapter 1 Introduction to SystemJ

1.1 Software Systems

Today's and emerging embedded and cyber-physical systems, which are mostly software based, are typically very complex and highly dependable and as such require rethinking of the approach to their design. With an increase in scale as well as heterogeneity of hardware platforms used to implement embedded systems, the designers need to employ development methodologies that accelerate their design process while guaranteeing the quality of the designs. Traditional ways of developing programs using low-level abstractions for interacting with hardware resources and employing operating system services further complicates the problem and hinders designers from understanding the system behaviours as a whole. Methodologies that naturally support decomposition of system functionality into hierarchically organised constituent parts, which execute concurrently, and use behavioural (algorithmic) descriptions are becoming increasingly important. Also, embedded and cyber-physical systems, which interact with their physical environment where the time dimension has to be taken into account as a part of a system functionality, are implemented by a new class of systems, software systems.

Unlike ordinary programs, software systems consist of many parts that have complex timing relationships and require sophisticated mechanisms for the description of their operation, which enable composition of bigger systems from the simpler parts, based on formal approaches that support analysis of system's operation using formal and automated methods. Consequently, they also require new design approaches and design languages that differ from the traditional general purpose programming practice embodied into the existing programming languages. These design approaches and languages are typically wrapped into complex design flows with many elements that allow structuring and analysis of system features before its deployment.

We informally define a software system as a collection of software modules connected each with the other through their input/output interfaces so that the system can communicate with its external environment through the system inputs and outputs and map system inputs onto the outputs through a series of transformations performed within interconnected modules and can explicitly include time dimension and time dependencies. The mappings of inputs to the outputs are predominantly in the form of behavioural (algorithmic) descriptions and can be augmented by the use of traditional programming techniques. A software system, when run on an execution platform, allows interaction with its environment, which may be a physical world, man-made systems or other software systems.

The languages for developing software systems should allow designers to easily capture their main characteristics, from structural and behavioural point of view, which include spatial and temporal properties, concurrency and hierarchy, as well as reactivity to the events coming from the systems immediate environment or from other parts of the system. In the following sections we elaborate these features of software systems in more details in order to come to our central theme of SystemJ language as the system-level design/programming language.

1.1.1 Spatial and temporal properties

When building a software system, designers should be able to set clear spatial and temporal boundaries and relationships among software modules. On one hand, spatial boundaries

between modules enable designers to focus on designing small and manageable system components where the scope of their working environment and visibility of shared resources from other components are clearly specified and confined in a managed way. Such approach significantly reduces chances of creating software bugs, which may not easily be found in the subsequent design process. On the other hand, temporal properties define the ability of software modules to cooperate with each other by establishing timing relationships and satisfy certain timing properties specified by the use of language. If we look, for example, to a typical synchronous digital circuit design, a clock signal is used to synchronise different parts of the circuit and establish timing relationships between those parts. It provides a deterministic relationship and enable communication among the components in the circuit, and exchange information with the external environment. In outset we propose that a similar approach is required and possible in software domain, described by operational rules and principles, which are formally defined to avoid potential ambiguities when implementing a software system. Establishment of both spatial and temporal boundaries between software modules must be supported on the system specification level and built in the language used for system design.

1.1.2 Reactivity

A reactive system is a system that continuously captures input events from an environment, perform internal computations based on those inputs, and generates (1) output events and data as a result of computations and transformations of inputs and (2) intermediate results kept in the system states. Today's embedded systems are mostly reactive; they observe the changes of their environment, make the decisions and generate actions accordingly. As a result, there are a number of important properties that designers need to consider when designing reactive systems:

1. *System's ability to capture input events.* Every software module designed to perform a specific task may become unresponsive to input events for certain period of time depending on their design, input capture mechanism, and operating speed (e.g. clock frequency). When a system misses inputs from the environment, it may completely fail or its quality of service is degraded, which may discomfort the users or produce a poor quality of results, e.g. streaming of video or sound, or even cause harm if the system is safety-critical. In extreme cases such misses may result in serious system malfunction that can harm people, e.g. in a car automatic braking system or loss of the control of industrial machines, autonomous vehicles and robots.
2. *System's ability to react on input events.* After capturing an input event from the environment, a reactive system should perform the operations that result in an update of the system's internal state. There should be a unique path to reach a certain state based on a sequence of input events received from the external environment. Such properties define the reactivity and determinism of a system operation, which are crucial for ensuring the system's correct operation and integrity. Therefore, it is important that designers can trust the developed software system and employ a well defined verification and validation methodology to reason about the system's ability to capture input events and react on their occurrence during its service.

1.1.3 Concurrency and hierarchy

Embedded software systems are highly concurrent in nature, described by many simultaneously operating and communicating behaviours implemented as software modules; they need to continuously react on events coming from the different sources in physical environment and require to serve those events in a timely manner, within pre-specified timing deadlines, thus being real-time systems. Traditionally, many embedded systems are

programmed in a very conservative fashion where algorithms and control-flow are designed sequentially in a single threaded loop that emulates the real-life concurrent/parallel operations. Designers need intuitive and highly flexible mechanisms to manage concurrent behaviours of a system that abstract away classical programming burdens such as avoiding deadlocks, orderly and safe sharing of resources, thread communication and synchronization, and data coherency. Hierarchical relationship among the concurrent behaviours is also an important feature when designing a system of high complexity. It allows system designers to decompose a system into software modules of closely related but simpler functionalities and then integrate them at the later design phase; this also plays an important role in creating fault-tolerant software systems. For example, actor based systems [1] implement fault handling supervisor strategy based on hierarchy of concurrent behaviours that propagates faults until they can be resolved by higher-level of intelligence.

One can start modelling and implementing software systems that exhibit aforementioned characteristics using traditional programming languages such as C/C++ or Java that are originally aimed at implementation of sequential algorithms and data abstractions typical for general purpose computing. However, designers require more in the form of tools that support designing software systems, which are beyond standard programming language and overcome their limitations. Significant efforts have been done in the research community toward the development of languages based on a formal foundation that enable correct-by-construction program design. These languages are equipped and underlined with a formal model of computation (MoC) that defines the execution model semantics and relationships of constituent parts of the program, thus allowing designers to reason about correctness of the software system by using automated procedures and tools. One of the active research directions of system-level languages and their formal model of computation is based on *Globally Asynchronous Locally Synchronous* (GALS) MoC, which extends another formal synchronous reactive MoC [2] with asynchronous parallelism, and suits much larger range of embedded and cyber-physical software systems. In this document, we present a GALS language SystemJ [3] as an answer to aforementioned questions and issues in designing complex software systems.

1.2 SystemJ Approach to Designing Software Systems

SystemJ can be considered an extension of synchronous reactive (SR) languages such as Esterel [2], Lustre [4], and Signal [5] to enable modelling more complex software systems that have synchronous subsystems as their constituent parts. SystemJ has all the features to model software systems using synchronous paradigm as well as the ability to further extend the system to globally asynchronous domain. Synchronous reactive MoC is very good at capturing behaviours of a system or its part and describing the operations that link inputs coming from the system's environment with the outputs generated as the reaction of the system to the environment. The synchronous part of the SystemJ language abstracts communication and interaction of the system with the external environment using the objects called *signals*. The signals are captured and emitted at the boundaries of discrete instants of time called *ticks*. In the synchronous parts of SystemJ program, time can be thought of as a sequence of ticks at which all the constituent components, called reactions, which are in the same synchronous domain, are triggered to execute and communicate with each other. The emitted signals are broadcasted to all reactions that may use them if needed, or just ignore them if not needed.

An ideal synchronous reactive system runs infinitely fast so that its environment cannot observe any delays caused by internal computations of a system, i.e. computations required to make a state transition. Therefore, generation of output events appears to be instantaneous to the environment. This is based on so called *synchrony hypothesis* [2]. The version of SystemJ

presented in this document uses a slightly different mechanism where a reaction on events received from the environment does not appear in the same logical tick of time but in the subsequent, next, tick, effectively being delayed by one tick, as will be explained in more detail in the next chapter.

SystemJ further extends the synchronous paradigm with asynchronous execution and communication model that allows system designers to model a system using a collection of loosely coupled synchronous domains. Therefore, a system designed in SystemJ follows GALS formal model of computation. In addition, SystemJ took a practical approach to link the language with a traditional programming language Java. This allows almost arbitrary embedding of Java statements in SystemJ program, so that complex algorithms and data abstractions can be used to describe the data computations and their mixing (interleaving) with the native SystemJ, called kernel statements, which are more natural for describing control-dominated behaviours, reactivity and concurrency. On the top system level a collection of loosely coupled synchronous domains execute asynchronously with each other, and we call them *clock-domains* (CDs). Each CD is driven by its own logical clock and its ticks, where clocks are independent each of the other, thus being asynchronous with each other. In SystemJ, two CDs running at their own speed can safely exchange data using CSP-style [6] channels. While CDs could communicate using signals, synchronous communication using signals is unreliable to exchange data between reactions belonging to different CDs due to mutually unrelated clocks and notion of time. Instead, channel allows CDs to ‘block’ until the other communicating CD is ready to exchange data. More details on how to write reactions, CDs and build a whole SystemJ systems will be explained in the next chapter.

Chapter 2 SystemJ Language – Semantics, Syntax and Programming

2.1 Design Entities

SystemJ programs, also referred as software systems (or just systems), are composed of components that run concurrently with each other. A graphical and intuitive representation of an example SystemJ program is shown in Figure 2.1. The program consists of two types of design entities:

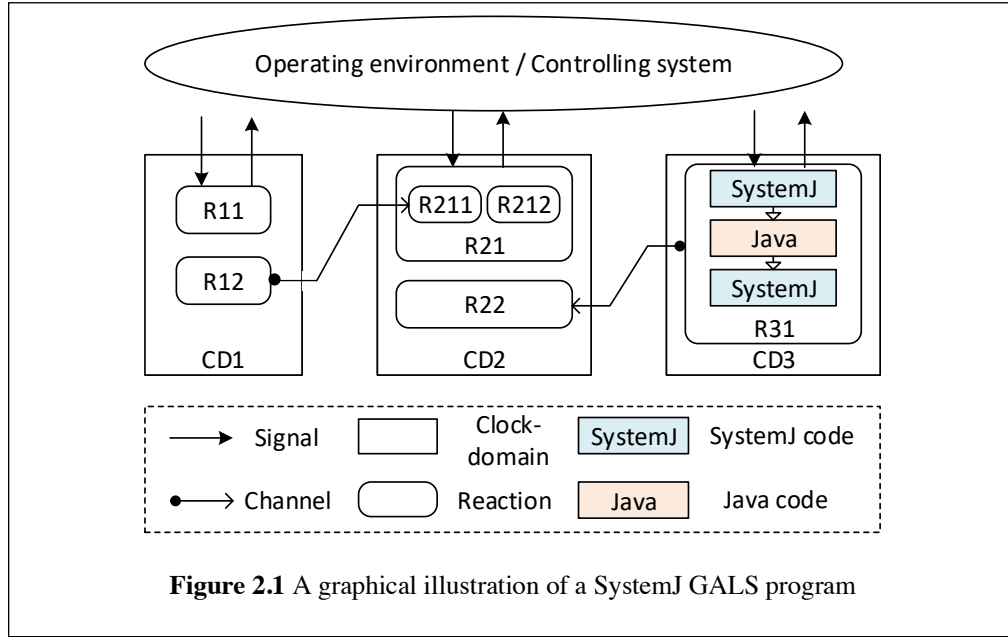
1. *Clock-domains* (CDs): They appear at the top level of design hierarchy. Each CD is a container for one synchronous (clock) domain hence each of them is driven by its own logical clock represented by a sequence of ticks ordered by time when they occur. In the example in Figure 2.1, there are three clock-domains that are called CD1, CD2 and CD3, running asynchronously with each other. It should be noted that these CDs can only be instantiated on a top-level of a system hierarchy, hence they represent the *globally asynchronous* part of the system.
2. *Reactions*: They appear as the basic design entities within the clock domains and can fork any number of other reactions, which are SystemJ threads, forming a parent-children relationship in a hierarchical tree structure with the common root. All these reactions belong to the same CD and execute computations that belong to one tick at a time; when all reactions execute a tick of their computations, they can proceed to the next tick. In the example in Figure 2.1, CD2 has two child reactions R12 and R22 where R21 forks two extra children called R211 and R212.

As we already introduced in Chapter 1, SystemJ provides deterministic communication mechanisms for exchange of data between reactions using two abstract objects called *signals* and *channels*.

1. Signal is the only means of communication among the reactions in a same CD. Signal is a tuple $S = (s_\gamma, s_v)$ where:
 - a. $s_\gamma \in \{+, -, \perp\}$ is a status of a signal, whether its status can be present, absent or unknown, respectively.
 - b. $s_v \in \{x \mid x \in J_{obj} \cup \{\perp\}\}$ is a value of a signal where J_{obj} is a set of all possible instances of any Java object.

Informally speaking, a signal consists of a status and a value at any logical tick. When the program execution starts, every signal is initialized with its status to unknown ($\perp$). Once emitted, the corresponding signal's status is set to true (+) and is valid for only one tick instant. The status of the signal is set automatically to false (−) in the next tick instant unless it is re-emitted in that tick. Signals can be emitted along with a value of type of any Java object. Unlike signal status, an emitted signal value persists across ticks until it is over-written by another emission of the same signal with a different value. Like the statuses, signal values are initially unknown ($\perp$) upon their initialization. Effectively, an emitted signal is broadcasted within a clock domain and its status and value are available for one tick to all reactions within the clock domain. Signals are also used for communication between CDs and the external environment;

they allow the exchange of data between a CD and the environment and thus they are also called interface signals. Signals received from the environment are commonly called input signals and those generated to the environment are called output signals.



Obviously, signals emitted through outputs from one CD can be captured through inputs of other CDs, however there is no guarantee that it will happen if the ticks of clock domains are not aligned, which is the main assumption of mutual asynchronicity of CDs.

2. *Channel* is the only reliable means of communication between two reactions in different CDs. As explained earlier, status of signals lasts for only a single tick; they are a standard way of communication mechanism between reactions that are synchronised with a same tick. However, as soon as reactions do not share a logical clock for their execution, signal communication becomes unreliable. Channels solve this problem by providing an abstract way of synchronising reactions that run asynchronously in two different CDs. We call this synchronisation mechanism rendezvous because both communicating reactions must agree on the exchange of data, effectively implementing full handshaking protocol and blocking until the exchange is completed successfully. This effectively means that two independent clocks of two CDs must synchronise at some ticks of their execution in order to enable the exchange of data. Upon completion of data exchange, the clocks become independent each of the other again. Channel communication is a *point-to-point* through the input and output interface port of the channel, respectively, performed by using *send* and *receive* statements, which exchange a value of any Java object (J_{obj}) in a reliable manner. Send statement is used by the sending reaction and receive statement by the receiving reaction. Formally, channel is a tuple $C = (c_{\gamma 1}, c_{\gamma 2}, c_{p 1}, c_{p 2}, s_v)$ where:

- a. $c_{\gamma 1}, c_{\gamma 2} \in \mathbb{B}$ are the statuses of input and output channel ports, respectively, for implementing conventional four-phase handshake protocol [7].
- b. $c_{p 1}, c_{p 2} \in \mathbb{Z}$ are the statuses of input and output channel ports, respectively, for implementing preemptive four-phase handshake protocol [7].

- c. $s_v \in \{x \mid x \in J_{obj} \cup \{\perp\}\}$ is a value, which is exchanged between reactions in two different CDs over the channel C .

SystemJ designers compose a system based on these design entities and communication objects, which are fundamental elements for capturing behaviours of software systems of high complexity. In section 2.3., we introduce a set of SystemJ kernel statements used to describe the behaviours and structural relationships of the design entities and their interconnections.

2.2 Tick-Based Execution

Execution of SystemJ CDs is based upon and driven by a sequence of discrete logical clock instants called ticks: each CD is driven by its own clock. Once a CD is triggered by the tick, it captures a set of input signals being monitored from the environment, makes internal decisions and computations based on the captured signals, as well as internally broadcasted signals and recorded state of all reactions, and finally generates output signals as a response. Figure 2.1 illustrates examples of execution traces for the two CDs, CD1 and CD2, which both generate an output signal O based on the sequences of inputs received through input signals A and B . CD1 emits O when both A and B become present irrespective of the order of their occurrence. On the other hand, CD2 only emits O when A and B are present in the specific order, first A and then B . In the example, CD1 could emit O when B and A became present in the reverse order at ticks t_3 and t_7 whereas O was only emitted by CD2 at t_{10} when A and B were captured in the specified order (i.e. t_9 followed by t_{10}).

The clocks and their ticks for individual CDs are *unrelated* with each other, and are synchronisation points for within each CD. For example, there is no relationship between the occurrences of ticks for CD1 and CD2 in Figure 2.1. Consequently, this implies there is **no guarantee** that how many ticks will be generated for CD1 between any two consecutive ticks of CD2. Such restriction, however, can be relaxed by employing a scheduler that makes execution of CDs predictable, e.g. CD2 must run at least once for every four ticks of CD1.

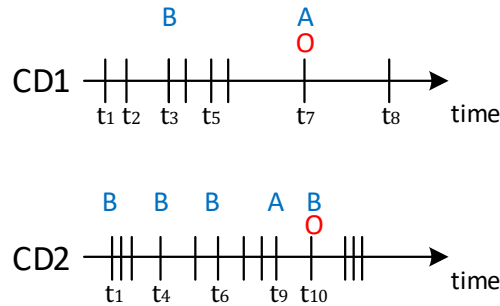


Figure 2.1 Execution traces of CD1 and CD2 where the inputs and output signal events are denoted by the blue and the red coloured characters, respectively.

At first glance, there are some similarities between CD ticks and a clock signal (physical clocks) such as in synchronous digital circuits. Indeed, both ticks and physical clocks are used to synchronise different parts of the system. However, unlike physical clock, CD ticks are ordered events where the duration between two consecutive ticks may vary depending on different factors imposed on the CD, for example, the amount of computations required to perform and the CD scheduling policy. For example, Figure 2.1 shows the time between ticks that varies for any two consecutive ticks.

2.3 SystemJ Kernel Statements

SystemJ is an imperative language similar to many other object-oriented programming languages. In addition to traditional object-oriented programming languages, the language provides a set of statements and constructs used to easily describe a complex control behaviours of today's embedded and reactive systems. Table 2.1 shows a list of the SystemJ kernel statements and first level descriptions of their meaning. SystemJ language defines structural translation, used for building an intermediate representation (IR) of the program [3], for each of these kernel statements. This intermediate representation is a graph, called *Asynchronous GGraph Code* (AGRC), which is the basis of code generation and formal verification of SystemJ programs.

Like many programming languages, SystemJ statements are separated by semicolons. Parenthesis can be used in usual way to define compound statements. The pause statement

Table 2.1 SystemJ reactive kernel statements.

Statements	Description
<code>p;q</code>	p and q in sequence
<code>pause</code>	Consumes a logical instant of time (defines a tick boundary)
<code>[input output] [type] signal ID</code>	Signal declaration
<code>emit ID[(expr)]</code>	Emitting a signal with a possible value
<code>[while(true) loop] p</code>	Temporal loop
<code>present(ID) p else q</code>	If signal ID is present enter p else q
<code>[weak] abort ([immediate] ID) p do q</code>	Preempt if signal ID is present
<code>[weak] suspend ([immediate] ID) p</code>	Suspend for one tick if signal ID is present
<code>trap(ID) p do q</code>	Software exception; preempt if exit(ID) is executed in p
<code>exit(ID)</code>	Throw an exception with a token ID
<code>p q</code>	Run p and q in lock-step parallel (synchronously)
<code>[input output] type channel ID</code>	Channel declaration
<code>send ID(exp)</code>	Send data exp over the channel ID
<code>receive ID</code>	Receive data on channel ID
<code>#ID</code>	Retrieve data from a valued signal ID or channel ID
<code>CD_name->{p}</code>	Defines p as a clock domain with name CD_name

explicitly indicates finishing of the computations of a reaction that belong to the current logical tick of a CD. If a CD is a single threaded, i.e. there is no other reactions currently running, the CD finishes its tick when the control flow hits the pause statement. In case if there are multiple reactions running, the CD finishes the current tick when all reactions hit their next pause statement. This denotes the end of the logical tick of the CD, which then can start with the execution of computations in its next tick. It is important to understand the fact that each CD communicates with the external environment via input/output signals at tick boundaries. Therefore, if any of reactions in a CD are unable to reach the pause statement, the CD is considered non-reactive and it is an incorrect SystemJ program. Input and output signals are declared in the interface section of a CD whereas internal signals are declared in the body part of a CD without the input and output identifiers. Signals can be either pure or valued. Pure signals at any tick have only binary status, present or absent (we call them also true and false, respectively), whereas valued signals can additionally carry a Java object instance as a value.

Signals can be emitted using the `emit` statement. Once emitted its status is set to true, which means that the signal is present, for only one logical tick and is broadcasted to all reactions in a same CD. The status is then set back to false, i.e. signal is absent, in the following tick whereas the signal value persists across ticks until it is overwritten by another `emit` statement.

Although each CD captures synchronous reactive behaviour, which is aimed at continuous response, SystemJ does not prevent designers to model transformational systems, which terminate after computing outputs for given inputs (e.g. compilers and simulators). On the other hand, reactive systems continuously react to inputs from the environment and are intended to run indefinitely. Such behaviours can be captured using the temporal loop statement `while(true)`; like the conventional while loop in other programming languages. SystemJ loops, however, require having at least one pause statement in its body so that allows a loop, and CD, to consume ticks, thus never being unresponsive.

There are several control flow statements in SystemJ that allow decision making and branching to different execution paths based on the statuses of signals. The `present(S) p else q` statement is like if-else in C/C++ or Java; however it evaluates the status of a signal and branches control flow to either p or q depending on the status of the signal. The `abort(S) p do q` statement allows monitoring of a signal and preempts control flow enclosed within the statement if the signal becomes present. The control flow then continues from the first statement in q. The `suspend(S) p` statement suspends (freezes) the enclosed control flow when the signal S is present for one logical tick. The `trap` statement has similar effect as the `abort` statement, where control flow in the enclosed statement gets preempted when a certain condition is met. Unlike `abort`, where the preemption depends on status of a signal, the preemption of `trap(T) p` happens when the control flow hits the `exit(T)` statement in p. Effectively, trap mechanism acts as a commonly known software interrupt, while `abort` acts as a hardware interrupt synchronised with a clock. The weak qualifier for the `abort` and `suspend` statements allows the program it encapsulates to finish a single tick before pre-emption. The immediate signal predicate, which can be combined with the weak qualifier, enables the program to react on the watching signals from the instant it enters p. Without the immediate, the program reacts on the signals after first tick has lapsed.

CDs spawn reactions using synchronous parallel operator `||`. For example, three reactions are spawns in the following statement: `p || q || r`.

Channels in SystemJ are declared in a similar fashion as signals. However, unlike signals, channels are always typed. When sending and receiving a message (Java object) over a channel, programmers use the `send` and `receive` statements, respectively. Once a message is received, it is retrieved using the hash operator `#`, e.g. `Object o = #C`. The same operator is used for retrieving a value of signals and serves as the interface from SystemJ statements to Java. Both `send` and `receive` statements are blocking, i.e. execution of a reaction will not continue until the exchange of the message sent over channel is completed. The exchange of message is governed by full handshaking between sending and receiving reaction in the process called rendezvous.

2.4 SystemJ Derived Statements

In addition to the presented kernel statements, SystemJ provides several derived statements that ease programming efforts and introduce useful behavioural patterns by removing boilerplate code in a program. These are the statements which are expanded into a sequence of kernel statements, i.e. they can be considered syntactic sugar. Table 2.2 shows a list of the derived statements. The `await(S)` statement puts control flow in an infinite temporal loop

until signal *S* becomes present, effectively halting the control flow by polling for a presence of the signal. This can be achieved using the `abort` statement enclosing the temporal loop as shown in the table. As explained previously, an emitted signal is present for only one logical tick. While some of the reactions may expect the emission of signals at certain ticks, it is not always the case and signals need to be continuously emitted until the other reactions become ready to realise the fact that the signal is present. Signals can be emitted continuously across ticks using the `sustain` statement. As shown in the table, the `sustain` statement is using the `while` statement that encloses the `emit` and the `pause` sequence. Unless a reaction is meant to sustain a signal for the life of the CD, the `sustain` statement is typically used along with other preemptive statements such as `abort` or `trap` to define a finite number of ticks the signal is emitted:

Table 2.2 SystemJ derived statements.

Statements	Expansion
<code>await([immediate] S)</code>	<code>abort([immediate] S){while(true)pause;}</code>
<code>sustain(S)</code>	<code>while(true){emit S; pause;}</code>
<code>abort p</code> <code>case (S1) do q1</code> <code>case (S2) do q2</code>	<code>abort(S1){abort(S2){ ... p ... } do q2 } do q1</code>
<code>halt</code>	<code>while(true) pause</code>
<code>loop</code>	<code>while(true) p</code>
<code>waitl (NUMBER (ms s))</code>	<code>trap(T){ long start = getTimeMs(); while(true){ if(getTimeMs() - start > NUMBER) exit(T); pause; } }</code>

```

abort(stop_emission){
    sustain A;
}

```

Reactions in a same CD communicate via a set of internal signals. Many control flow statements checking different signal statuses can complicate logic that may reduce code readability. This can be improved using the `abort-case` construct that has separate case statements ordered by their priority. For example, consider the following snippet of code that sustains signal *A* until either *I1* or *I2* becomes true:

```

abort(
    sustain A;
} case (I1) do { p }
   case (I2) do { q }

```

In this code, control flow remains inside the `while` loop in the `sustain` statement and jumps out to either *p* or *q* depending on the signals *I1* and *I2*. When both *I1* and *I2* are true in the same tick, control flow jumps to *p* as *I1* has higher priority than *I2*. Reactions can wait indefinitely using the `halt` statement, and `loop` is synonym of `while(true)`. All the

previously introduced statements that block control flow, e.g. `halt` and `await`, are using time expressed in terms of logical ticks. In fact, ticks are the only way of achieving awareness of time in SystemJ. In order to describe relationships to and achieve the real-time behaviours in a program, SystemJ provides `waitl` statement, which employs the execution platform specific function that returns the current wall-clock time available on the platform. For example, the following code generates a signal A with a period equal 200ms:

```
while(true){
  { sustain A; } || { waitl(100 ms); }
  waitl(100 ms);
  pause;
}
```

2.5 Writing Simple SystemJ Programs – Examples

After introducing the kernel and derived statements of the language, in this section we present the first steps of programming synchronous and GALS programs in SystemJ.

2.5.1 Synchronous programs

This section gives an introduction on how to write SystemJ programs using the statements presented in Section 2.3 and 2.4. The first program shown below is a synchronous program represented by a single CD that prints “Hello World”.

```
P1->{ System.out.println("Hello World"); }
```

Here, a CD called P1 is declared using the notation a CD_NAME followed by the arrow `->`. The actual SystemJ code is written in the block `{ }`. In this code, the CD P1 prints “Hello World” using Java method and immediately terminates in the same tick. We can explicitly insert tick boundaries using `pause` as shown below:

```
P2->{
  System.out.println(1);
  pause;
  System.out.println(2);
  System.out.println(3);
  pause;
}
```

Here, CD called P2 prints 1 in the first tick, and 2 and 3 in the second tick. It is important to understand that although there is an additional print statement for the CD to execute in the second tick, logical time requires two ticks before terminating this CD regardless of complexity of computation during each tick. Interaction between a CD and the external environment done via I/O signals is shown in the following example:

```
P3(input String signal A; output signal B;)->{
  await(A);
  System.out.println("Received " + #A);
  emit B;
}
```

When P3 has I/O interface for signals or channels, they are declared inside the parenthesis after the CD name. Here, there are one input signal A of type `String` and one binary (pure) output signal B. The CD's execution is halted by the `await` until the presence of signal A. The value of the received signal is then printed in the following print statement and an output is emitted back to the environment via B. As shown above, SystemJ allows designers to write Java code that mixes naturally with the SystemJ kernel statements. This greatly helps describing very complex systems that are heterogeneous in nature and require to perform both control and data rich/dominated operations.

Similarly, a CD can have a channel to enable communication with another CD. If a CD sends data over channel it has to have an output channel port declared in its I/O interface. Then, that channel port can be used in send statement to send the data received over signal A from the environment to another CD within a larger program:

```
P3(input String signal A; output signal B; output String channel C)->{
    await(A);
    System.out.println("Received " + #A);
    emit B;
    send C(#A);
}
```

CDs can fork synchronous parallel reactions when multiple operations need to be done in parallel (concurrently). In SystemJ, reactions enable local synchronous concurrency within a CD and execute in lock-step like synchronous blocks in a digital circuit that are driven by the common physical clock. For example, consider the following code where three reactions are forked upon the start of CD execution:

```
P3(output signal A, B; output signal O;)->{
    // Reaction 1
    pause; // 1st tick
    emit A;
    pause; // 2nd tick
}
||
// Reaction 2
    pause; // 1st tick
    emit B;
    pause; // 2nd tick
}
||
// Reaction 3
    await(A && B); emit O; }
}
```

Upon its start, P3 immediately forks three reactions where two of them emit A and B, respectively, and the third one waits on the presence of the both signals in the *same tick* in order to emit signal O. Normally, traditional programming languages run these threads with support of the language runtime environment such as operating systems or execution platforms (e.g. JVM). Problem with traditional threads is that programmers need to be aware of non-deterministic nature of the thread's model of computation [8], which can create many well-known problems when using threads. In SystemJ, programming concurrent behaviours within a CD is much easier because the execution semantics of reactions is deterministic. For example, the pause statements divide the execution of the first two reactions into two ticks. Since

reactions run in lock-step and parallel, we know that both A and B are emitted during the second tick. This fact is propagated to all other reactions within the same CD and as a result, the third reaction emits O. Although statuses of emitted signals last for only one tick, we can easily deduce that the third reaction will never miss the signals as it is guaranteed by the semantics of the synchronous reactions in SystemJ.

The only means of communication among reactions within a CD is by using signals; SystemJ prohibits the use of Java variables to pass data between reactions. In fact, any variables declared inside a reaction are *only* local to that reaction:

```
P4->{
  int i = 0;
  { int i = 1; } || { int i = 2; } || { int i = 3; }
  System.out.println(i);
}
```

Here, there are four reactions in total including the top-level reaction which declares variable *i* and then forks other three reactions. As a result of executing this CD, 0 will be printed as *i* is only local to the reaction where it is accessed. The other three reactions have their own local variables with the same name *i*, visible only within those reactions.

Now, we can introduce another important feature of signals emitted within a CD in SystemJ, which is related to the time (tick) in which emitted signals become visible with the present status. In SystemJ, broadcasted signals are by default only visible in the tick that follows the tick in which they are emitted. This is different to other synchronous languages such as Esterel [2] where signals are visible instantaneously in the same tick in which they are emitted. For example, see the following code snippet where signals B and C are emitted based on the condition that the signal A is present:

```
P5-> {
  signal A,B,C;
  { emit A; } // R1
  ||
  { // R2
    present(A) {
      emit B;
    }
  }
  ||
  { // R2
    pause;
    present(A) {
      emit C;
    }
  }
}
```

Unlike `await`, the signal test for the `present` statement is instantaneous, i.e. does not consume ticks. In the code above, reaction R2 tests presence of A in the same tick when it is emitted, i.e. there is no `pause` statement that separates those two statements. However, as previously explained, this test fails as visibility of signals is *delayed* to the next tick. As a result, this program does not emit B, as the presence of A is checked in the same tick in which it is emitted, but it emits C in reaction R3 where the `present` statement checks A in the following tick.

2.5.2 GALS programs

SystemJ synchronous programs, which are used to design CDs, are used as a basis and components for the extension to the GALS programs. A GALS program consists of multiple CDs that run asynchronously, driven with their own mutually unrelated logical clocks. Informally, you can think of individual CDs as traditional threads in GALS program. In GALS program, designers should not think of a program with any assumption of the speed of individual CDs: their speed, execution order, and scheduling priority are not formally defined. This means scheduling policy of CD can even be dynamically changed during runtime if required by the underlying runtime system. See the following example that executes two CDs asynchronously:

```
P6(output signal A;)->{
    emit A;
    pause;
}

P7(input signal A;)->{
    await(A);
    System.out.println("received A");
}
```

In this example, P6 emits A, which is captured in P7 at the `await` statement. Here, P6 sets A to true for one tick. However, ticks in P6 are unrelated with the ticks in P7; it is possible that runtime system always run two ticks for P6 before running P7 and as a consequence P7 will never capture this signal. On the other hand, if the runtime system “decides” to run P6 and P7 cyclically, A is always received by P7 *for this specific execution order*. Signal communication between CDs is therefore unreliable and it is not deterministic mechanism in GALS context. In SystemJ, a deterministic message passing mechanism is used for communication between reactions in two different CDs. Such mechanism is encapsulated in a high-level communication object called *channel* as shown in the code below:

```
P8(output String channel C;)->{
    send C("World");
}

P9(input String channel C;)->{
    receive C;
    System.out.println("Hello "+#C);
}
```

This example replaces signals with channels in the previous example. Here, P8 sends a Java String object “World” to P9 where it prints the received message “Hello World” on a terminal. In SystemJ, channel is unidirectional and point-to-point, which means reactions cannot send or receive on a same port concurrently (in parallel). For example, the following SystemJ code is incorrect as P10 tries to send multiple values on a same channel port:

```
P10(output int channel C;)->{
    { send C(1); } || { send C(2); }
}

P11(input int channel C;)->{
    receive C;
    System.out.println(#C);
}
```

However, it is possible to write multiple `send` or `receive` statements if they are never executed in mutually parallel reactions and in the same tick:

```
P12(output int channel C;)->{
    send C(1);
    { send C(1); } || { /* empty reaction */ }
    send C(1);
}
P13(input int channel C;)->{
    receive C;
    if(/* some expression */) {
        /* Do some work */
        receive C;
    } else {
        /* Do some other work */
        receive C;
    }
    receive C;
    System.out.println(#C);
}
```

2.6 SystemJ Helper Constructs

Apart from the kernel and derived statements, SystemJ also provides a number of helper constructs that enrich its capabilities. For example, designers can import Java classes using Java's `import` statement:

```
import java.util.ArrayList;

P14(output ArrayList signal A;)->{
    ArrayList list = new ArrayList();
    list.add(123);
    emit A(list);
}
```

In this example, `java.util.ArrayList` class is imported from the standard library and it is used to declare signal `A` of type `ArrayList`. The object instance `list`, which contains an integer number 123, is emitted as a signal value for `A`. Designers can reuse a block of code by declaring the block as a reaction and giving it a name:

```
reaction add(int x, int y: output int signal R;){
    emit R(x + y);
}

reaction capture(:input signal A, input signal B) {
    {await(A);} || {await(B);}
}

P15(input int signal I1, I2, I3, I4; output int signal O1, O2;)->{
    capture(I1,I2) || capture(I3,I4)
    add(#I1,#I2 : O1);
    add(#I3,#I4 : O2);
}
```

When the designers declare named reactions they have to use the following syntax:

```
reaction NAME(<list_of_constants> : <list_of_IO_signals_and_channels>)
{
    /* SystemJ code */
}
```

Reactions have two types of parameters: one is a list of constants, represented by their identifiers/names, and the other one is a list of IO signals and channels reaction operates on. It should be noted that those constants cannot be assigned with a new value inside the reaction blocks. When reactions are declared, they can be invoked in a CD as shown in the above code. Reactions can be invoked by using their name. For example, P15 spawns two instances of capture reaction by passing input signals I1 and I2 to the first instance and I3 and I4 to the second instance. This will result in total four reactions to be spawned as two additional unnamed reactions are spawned by the capture reaction. Reactions can also be executed like a procedure as shown in the example above where the reaction add adds two signals values and emits the result via the output signal R.

SystemJ provides bundling of related CD designs via packages. For example, suppose a designer has developed reactions that model smart devices for home automation application. We can group such reactions under the namespace called “smarthome”. In SystemJ, this can be achieved by declaring the `syspackage` statement in the beginning lines of the source code:

```
// filename: homel.sysj in a directory $PROJECT/smarthome
syspackage smarthome;
reaction LightControl( :input int signal Intensity; output int signal
O; )
{ /* Implementation */ }
reaction MotionSensor( :input Motion signal InMotion;)
{ /* Implementation */ }
reaction DoorControl( :input signal Closed,Opened; output int signal
TriggerAlarm;)
{ /* Implementation */ }

// Instantiating reactions
LivingRoomLighting(..)->LightControl
Security( input Motion signal InMotion, input signal
DoorClosed,DoorOpened;
    output signal Motion; output signal TriggerAlarm;)->{
    MotionSensor(:InMotion) ||
    DoorControl(:DoorClosed,DoorOpened,TriggerAlarm)
}
```

Designers can instantiate these reactions in the same file as shown above, where LightControl reaction is instantiated as a top-level CD called LivingRoomLighting, and both MotionSensor and DoorControl are instantiated as a reaction within a CD called Security. These reactions can also be imported into a SystemJ source code in other packages:

```
// filename: home2.sysj in a directory $PROJECT
sysimport smarthome.*;

OfficeLighting(..)->LightControl
Office Security( input Motion signal InMotion, input signal
DoorClosed,DoorOpened;
    output signal Motion; output signal TriggerAlarm;)->{
    MotionSensor(:InMotion) ||
DoorControl(:DoorClosed,DoorOpened,TriggerAlarm)
}
```

Here, reactions `LightControl`, `MotionSensor` and `DoorControls` are imported from the file located in `./smarthome`. Fully qualified names for CDs are identical to the notation used in Java. For example, designers can import only the `LightControl` which is in a package `smarthome`. In this case, the `LightControl` should be imported using its fully qualified name as follows:

```
sysimport smarthome.LightControl;
```

Currently, the top-level package is in a directory where the SystemJ compiler is invoked. For example, if the compiler is invoked in a directory `$PROJECT`, this directory becomes a top-level package. Note that designers cannot create a namespace for the top-level package and it is automatically named as `“.”`. In addition, CDs in the `“.”` namespace cannot be imported into other packages. On the other hand, all directories that reside in the top-level directory become named package. For example, the previous CDs in the directory `$PROJECT/smarthome` are the members of the `smarthome` package given that the package is declared using the `syspackage` statement in the source code.

Chapter 3 Designing Software Systems

In this chapter, we introduce a number of examples and descriptions of simple SystemJ programs/systems. First, we illustrate in more details how to write a simple sequential reactive program/system. Composing multiple sequential programs using synchronous paradigm is presented next. Then we introduce GALS and describe how its behaviour differs from a pure synchronous program.

The rest of the section is devoted to the descriptions of two concrete SystemJ examples. First, we introduce a purely synchronous program and demonstrate how SystemJ can easily capture concurrent behaviours in a synchronous setting. Then, we extend the example to a GALS program and describe how programs can be partitioned into groups of synchronous islands, which are mutually asynchronous clock domains.

3.1 Designing Synchronous Systems

3.1.1 A Sequential Reactive Program

The simplest form of a SystemJ program is a sequence of statements grouped in a reaction as shown in the code below, where the execution blocks inside the `while` loop until signal `A`, which is used to preempt the execution of the loop, becomes present. The reaction then emits `O` before it terminates, i.e. control flow reaches the end of the final curly bracket.

```
{
  abort(A) {
    while(true) {
      pause;
    }
  }
  emit O;
}
```

This reaction can be simplified using the `await` statement as explained in the preceding chapter:

```
{
  await(A);
  emit O;
}
```

To connect this reaction to the environment, we declare a CD with an input and an output signal port that match with the names of signals used in this reaction:

```
SequentialProg(input signal A; output signal O;)->{
  await(A);
  emit O;
}
```

Reactive systems most often run indefinitely executing the same type of behaviour repeatedly. For example, the system repeatedly checks for presence of signal A and after the signal becomes present emits signal O. We can make this program to repeat using an outer-most `while` loop as shown below:

```
SequentialProg(input signal A; output signal O;)->{
    while(true){
        await(A);
        emit O;
    }
}
```

We have successfully created our first SystemJ reactive system. This is a system that reacts to an input signal A and generates an output signal O as a response. The behaviour of this program is quite simple: its only role is to inform the environment via signal O that it has successfully received an input A. Let us introduce additional signals that trigger different operations:

```
SequentialProg(input signal A,B; output signal O1,O2;)->{
    while(true){
        await(A || B);
        present(A){
            emit O1;
        }
        present(B){
            emit O2;
        }
    }
}
```

Now the program emits either O1 or O2 based on the signal it has received. In this example, the program waits for the presence of signal A or B, which is expressed using the `await` statement with the logical OR (`||`) operator applied to the statuses of these signals. The emission of either O1 or O2 is described using two `present` statements that test the presence of A and B, respectively. In case when the program captures both A and B simultaneously (i.e. in the same tick), it emits both O1 and O2. Let us consider a case when a designer wants to make signal A with a higher priority over signal B such that only O1 would be emitted when both A and B were captured in the same tick. Such behaviour can be easily captured in SystemJ using nested `abort` statements:

```

SequentialProg(input signal A,B; output signal O1,O2;)->{
  while(true){
    abort(A){
      abort(B){
        halt;
      } do {
        // Signal B handler
        emit O2;
      }
    } do {
      // Signal A handler
      emit O1;
    }
  }
}

```

This program blocks at the `halt` statement where it gets preempted and enters one of two signal handlers when either A or B becomes present. When both A and B are present, the signal A handler gets higher priority as the `abort` statement for signal A is enclosing the `abort` statement for signal B. Alternatively, this program can be rewritten as the follows, which has identical behaviour as the previous example:

```

SequentialProg(input signal A,B; output signal O1,O2;)->{
  while(true){
    abort {
      halt;
    } case (A) do { emit O1; }
    case (B) do { emit O2; }
  }
}

```

In addition to the emission of the status of signals to the environment, we can also provide additional information with the signals by attaching a value. For example, we can emit a total number of events captured via A and B using a simple integer counter (note that we have added a Java primitive type `int` for signals O1 and O2):

```

SequentialProg(input signal A,B; output int signal O1,O2;)->{
  int aCount, bCount;
  while(true){
    abort {
      halt;
    } case (A) do { emit O1(aCount++); }
    case (B) do { emit O2(bCount++); }
  }
}

```

Once signals are emitted, their status becomes true (present), which lasts for only a single tick. For example, one might try to modify the program as follows to test the presence of signals O1 and O2 using the `present` statement:


```

SequentialProg(input signal A,B; output int signal O1,O2;)->{
  int aCount, bCount;
  while(true){
    abort {
      halt;
    } case (A) do { emit O1(aCount++); }
      case (B) do { emit O2(bCount++); }

    present(O1){ System.out.println("I see O1"); }
    present(O2){ System.out.println("I see O2"); }
  }
}

```

However, this program never captures O1 and O2 as it checks for presence of these signals in the same tick they are emitted. In fact, signals emitted by the SystemJ program only become visible to other reactions in the same CD in the following tick. To make this program capturing O1 and O2, we need to insert a pause between the emit and present statements:

```

SequentialProg(input signal A,B; output int signal O1,O2;)->{
  int aCount, bCount;
  while(true){
    abort {
      halt;
    } case (A) do { emit O1(aCount++); }
      case (B) do { emit O2(bCount++); }

    pause;

    // The following present statements can capture O1 and O2 in the
    // next tick

    present(O1){ System.out.println("I see O1"); }
    present(O2){ System.out.println("I see O2"); }
  }
}

```

Such a delay in the visibility of emitted signals can further increase response time of a CD in the presence of more synchronous concurrent behaviours in a CD.

3.1.2 Designing Synchronous Systems – An Industrial Automation Example

In this section, we demonstrate how to build a concurrent, synchronous system in SystemJ using industrial automation control example [9], where four pushers are used to move a workpiece from its initial (arrival) position to the final (destination) position. The controller consists of four synchronous reactions, each of which is a pure sequential program, controlling the movement of the workpiece. A source position is identified by the workpiece arrival sensor. After each move, the pusher that moves the workpiece to the destination position must retract to its initial position. Figure 3.1 shows an overview of the system with its graphical representation. As one can see in the figure, there are two vertical cylinders (CylV) and two horizontal cylinders (CylH) which together, in coordination, move the workpiece to one of the grey shaded destination positions. Each cylinder is controlled by its own controller reaction that

Table 3 The cylinder controller interface

Signal	Direction	Description
WPINPLACE	Input	Present when the workpiece is in front of the cylinder
CYLATSTART	Input	Present when the cylinder is at the initial position
CYLATEND	Input	Present when the cylinder is fully extended
MYTURN	Input	Present when a token is received
PUSH	Output	The cylinder extends as long as PUSH is present
POP	Output	The cylinder retracts as long as POP is present
YOURTURN	Output	To deliver a token to the next cylinder

has a set of input and output signal ports as shown in Table 3. The system shown in Figure 3.1

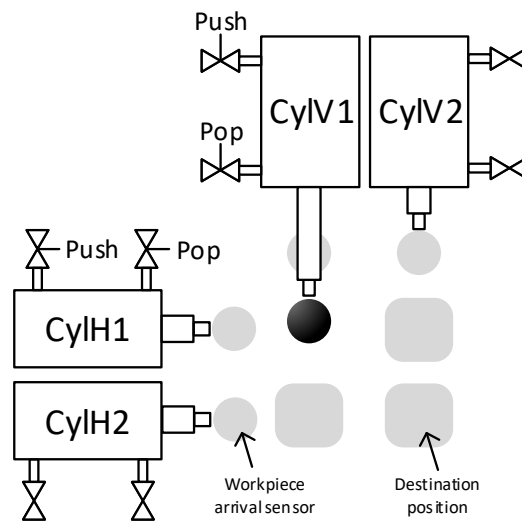


Figure 3.1 4-Cylinders System

requires additional coordinating logic to avoid potential collisions between the pushers. In this case, we choose a ring-token approach [10], where a cylinder fires only when a workpiece has arrived *and* it holds the token.

```

1  reaction CylinderController(boolean INIT :
2      input signal WPINPLACE, input signal CYLATSTART, input signal
3  CYLATEND,
4      input signal MYTURN,
5      output signal PUSH, output signal POP, output signal YOURTURN) {
6
7      signal OVERRIDE;
8      if(INIT)
9          emit OVERRIDE;
10
11     while(true){
12         await(MYTURN || OVERRIDE);
13         present(WPINPLACE) {
14             abort(CYLATEND) { sustain PUSH; }
15             abort(CYLATSTART) { sustain POP; }
16         }
17         emit YOURTURN;
18     }
19 }

```

Figure 3.2 SystemJ implementation of the cylinder controller

Figure 3.2 shows the SystemJ code implementing the cylinder controller logic. We implement this controller as a reaction component so that the same code can be reused to instantiate controllers for all four cylinders. The reaction is named `CylinderController`, with the input and output signals as listed in Table 3. The program should start with one cylinder that has the token. We pass a Boolean value `true` to the cylinder starting with the token, which is indicated by the constant `INIT` (line 1). The controller checks if `INIT` is true and emits `OVERRIDE`, which will preempt the `await` statement (line 11) and check if the workpiece is in place (line 12). If the sensor detects the workpiece, the cylinder starts to push it until the cylinder is fully extended (line 13), then it retracts back to the initial position (line 14). Finally, the current cylinder passes a token to the next cylinder by emitting `YOURTURN` (line 16).

```

1 CylinderSystem(input signal PUSH1,PUSH2,PUSH3,PUSH4;
2                 output signal POP1,POP2,POP3,POP4;
3                 input signal WPINPLACE1,WPINPLACE2,WPINPLACE3,WPINPLACE4;
4                 input signal CYLATSTART1,CYLATSTART2,CYLATSTART3,CYLATSTART4;
5                 input signal CYLATEND1,CYLATEND2,CYLATEND3,CYLATEND4;)->
6 {
7     signal WIRE1,WIRE2,WIRE3,WIRE4;
8
9     CylinderController(true:WPINPLACE1,CYLATSTART1,CYLATEND1,WIRE4,PUSH1,POP1,WIRE1)
10    ||
11    CylinderController(false:WPINPLACE2,CYLATSTART2,CYLATEND2,WIRE1,PUSH2,POP2,WIRE2)
12    ||
13    CylinderController(false:WPINPLACE3,CYLATSTART3,CYLATEND3,WIRE2,PUSH3,POP3,WIRE3)
14    ||
15    CylinderController(false:WPINPLACE4,CYLATSTART4,CYLATEND4,WIRE3,PUSH4,POP4,WIRE4)
16
17 }

```

Figure 3.4 Composing sequential systems using synchronous paradigm

Now we have to instantiate four reactions using single-cylinder controller component from Figure 3.1. An overview of our 4-cylinder controller system is illustrated in Figure 3.3, where the rectangles with rounded corners are the reactions and the outermost rectangle indicates a CD consisting of those reactions. Other than MYTURN and YOURTURN signals, which are used to pass a token around the reactions, all input and output signals of the reactions are connected to one of the four pneumatic cylinders. If we assume that CyIH1 has a token initially, we can build this CD by composing the `CylinderController` reaction as shown in Figure 3.4. As one can see in the figure, we pass a Boolean value `true` to the first reaction we created (line 9) and `false` to the rest (lines 11, 13, and 15). We use internal signals `WIRE` to connect input and output ports of the cylinder controller, see how they are connected in the figure. All other ports of the `CylinderController` reactions are directly mapped to the corresponding input/output ports of the top-level CD `CylinderSystem`.

One main advantage of the pure synchronous systems is their determinism. In the example above, the communication among the cylinders is easily captured using signals thanks to their

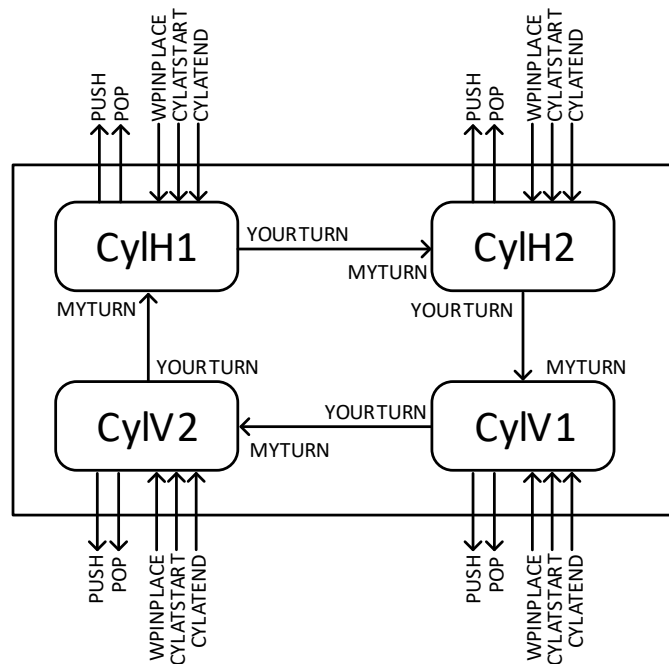


Figure 3.3 Single clock-domain solution of the 4-cylinder system

synchronous semantics and lock-step execution of the concurrent reactions. However, a pure synchronous approach is not suitable to describe large systems, especially if they are distributed and heterogeneous in nature. For example, synchronising a large number of reactions that execute on different computers over low-bandwidth network would be undesirable as those reactions would require to synchronise with each other quite frequently, e.g. at every CD tick. In this case, GALS approach to composition of reactions and CDs would be more appropriate where we can divide and group closely related behaviours (reactions) into synchronous island and let them communicate only when necessary through safe channels.

3.2 Designing Asynchronous Systems

We will re-use the 4-cylinder system from the previous section for demonstrating creation of a GALS controller. In this example, instead of using signals for synchronisation and communication between the reactions, we will instantiate a CD for each cylinder controller and connect CDs using channels. Figure 3.5 shows the GALS version of the 4-cylinder system where we replaced MYTURN and YOURTURN with input and output channels, respectively. There is a reaction (line 20-26) that receives a token on MYTURN and emits TOKEN_RECEIVED, which starts the cylinder operation in the reaction at line 9-18. The token is then passed to the next CD via YOURTURN at line 16. On the top-level of the SystemJ program/system, the modified reaction `CylinderController` is used to instantiate four CDs that control the cylinders, see lines 29-38. One thing to note here is that we need to specify which CD initially has a token when the system starts to operate. In SystemJ, we provide a shortcut `CD (. .) -> R` to instantiate the CD with the reaction R, i.e. R is a top-level CD. In this case, all input and output ports of the CD are automatically mapped to the interface of the reaction R. In order to change this mapping, we need to wrap the reaction inside a CD where we would declare each port normally, see lines 29-34. If we use the shortcut notation, we cannot pass the `INIT` parameter to the CD and it is assigned with an initial value as if we declare it as a Java class field without assigning a value. This is the reason that we did not use the shortcut notation for instantiating `CylH1` as we need to pass `true` to `CylinderController` reaction (line 32). All other CDs (`CylH2`, `CylV1` and `CylV2`) are instantiated using the shortcut notation where the `INIT` parameters are passed with `false`.

It is common practice in SystemJ to make separate reactions (e.g. lines 20-26 in Figure 3.5) for receiving data from an input channel port and notify other reactions on the message using signals. In this way, reactions that perform control operation will not block and only react to a signal that indicates an arrival of new data (message) from another CD. Furthermore, data arriving from multiple sources can be merged, eventually after some processing, into a single message using signals and their values, effectively implementing data fusion. In Figure 3.5, `TOKEN_RECEIVED` captured at line 11 using the `await` statement starts the cylinder operation where the signal can be emitted by setting `INIT` (lines 7-8) or by receiving a token from another CD (lines 22-23).

Figure 3.5 shows only the abstract connections of CDs with other CDs and external environment but does not show the low-level details of how the used channels and interface signals are actually implemented. For this purpose, SystemJ employs a program *configuration file*, where designers specify the details of CD interconnections, which output and input channel

```

1  reaction CylinderController(boolean INIT :
2      input signal WPINPLACE, input signal CYLATSTART, input signal CYLATEND,
3      output signal PUSH, output signal POP,
4      input int channel MYTURN, output int channel YOURTURN) {
5
6      signal TOKEN_RECEIVED;
7      if(INIT)
8          emit TOKEN_RECEIVED;
9      {
10         while(true){
11             await(TOKEN_RECEIVED);
12             present(WPINPLACE) {
13                 abort(CYLATEND) { sustain PUSH; }
14                 abort(CYLATSTART) { sustain POP; }
15             }
16             send YOURTURN(1);
17         }
18     }
19     ||
20     {
21         while(true){
22             receive MYTURN;
23             emit TOKEN_RECEIVED;
24             pause;
25         }
26     }
27 }
28
29 CylH1(output signal PUSH; output signal POP;
30 input signal WPINPLACE,CYLATSTART,CYLATEND;
31 input int channel MYTURN; output int channel YOURTURN;)->{
32     while(true){ CylinderController(true:WPINPLACE,CYLATSTART,CYLATEND,PUSH,POP,
33                                     MYTURN,YOURTURN); }
34 }
35
36 CylH2(..)->CylinderController
37 CylV1(..)->CylinderController
38 CylV2(..)->CylinderController

```

Figure 3.5 Extending the cylinder controller to GALS

ports are connected each to the other, signal interface, etc. More details on the SystemJ configuration file are presented in the context of SystemJ runtime environment in Chapter 5.

3.3 Full Synchronous System Example - ABRO

ABRO is a simple yet the most instructive example for illustrating features of synchronous reactive language [2]. ABRO is a reactive program that waits for the presence of signals *A* and *B*, which are generated by the external environment. When both signals are present, the program emits *O* to the environment. The program resets to its initial state when another signal *R*, which preempts the current operation, becomes present. A finite state machine (FSM) capturing this reactive behaviour is shown in Figure 3.6(a). As one can see from the figure, there are four states in total (*S0*, *S1*, *S2*, *S3*) and eight possible transitions from these states. When the program starts, program first enters state *S0*, which is an initial state. From this point onward, program can make transition to *S3* with three possible routes that depend on the sequence of *A* and *B* generated from the environment: 1) *A* followed by *B*, 2) *B* followed by *A*, and 3) *A* and *B* simultaneously. In any of the four states, signal *R* resets the FSM back to the initial state (*S0*). As one may already noticed, the transitions in this FSM models concurrency of the ABRO where inputs can be generated by the environment simultaneously or in any arbitrary order. Figure 3.6(b) shows one example of the FSM trace. In this example, the FSM makes first transition to *S1* with the presence of signal *B*. Signal *R* makes the FSM reset to *S0* in the next tick. After capturing *B* and *A* in sequence, the system emits *O* during the state

transition from S1 to S3. The system emits another O after resetting its state back to S0 when A and B are present in the same tick.

SystemJ implementation of ABRO is shown in Figure 3.7. In this code, capturing of signals A and B are done using synchronous reactions as shown on line 4. Synchronous parallel operator that spawns two reactions, where each of them is waiting for the input signal using the `await` statements, makes sure that the control flow will wait for presence of both A and B in any order before the control flow can proceed to emit O (line 5). Upon receiving both signals, the CD blocks at the `halt` statement, which is translated to `while(true) pause`, on line 6. Preempting concurrent reactions in SystemJ is explicit and much easier than in many other programming languages where designers need to manually clean up resources in order to shutdown threads gracefully. In SystemJ, any ongoing behaviours, regardless how many reactions are currently active, can be preempted using the `abort` statement as shown in lines 3-7. In this example, CD preempts whenever signal R becomes present and control flow jumps to the statement which is followed by the `abort` statement (i.e. end of the loop body).

One of the major strengths of the SystemJ, as highlighted in the ABRO example, is hiding the complexity of system's behaviour through abstractions, such as concurrency, hierarchical composition, preemption, etc., using high-level and intuitive language constructs. It is still manageable for the designers to model systems like ABRO whose entire behaviour can easily be captured without too much effort. Nevertheless, the size of software systems can grow substantially and sometimes it would be unrealistic to capture their behaviour exhaustively

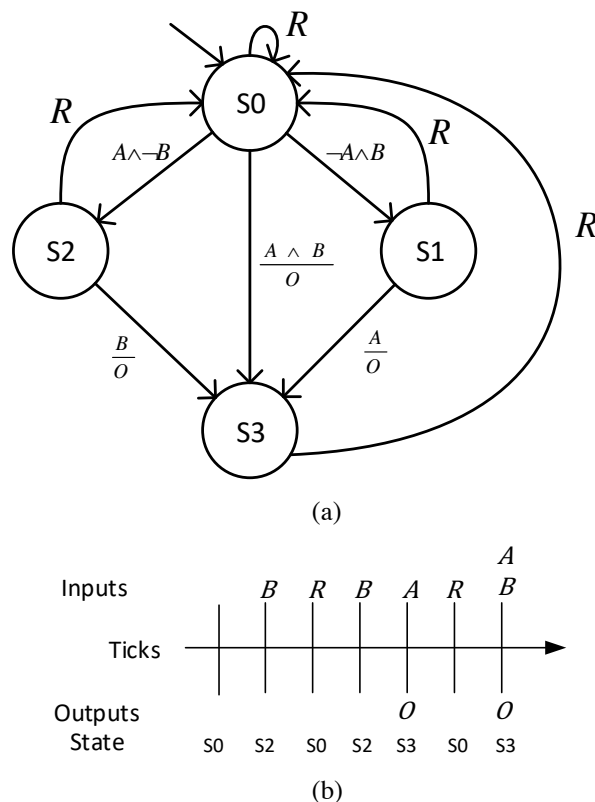


Figure 3.6 ABRO modelled in (a) a FSM and (b) an example of execution trace

```

1 ABRO(input signal A,B,R; output signal O;)->{
2     loop{
3         abort(R){
4             {await(A);}||{await(B);}
5             emit O;
6             halt;
7         }
8     }
9 }

```

Figure 3.7 ABRO in SystemJ

using the conventional approaches. With SystemJ, it is much easier to capture the system's overall behaviour where its formal model is automatically captured from the high-level description which can be used to both validate the system's behaviour and to produce executable code for deployment.

3.4 Full Asynchronous System Example - Producer Consumer ABRO (PCABRO)

PCABRO, which consists of three CDs, uses ABRO to describe two behaviours, a data producer and a data consumer, where the producer behaviour generates and sends messages to the consumer behaviour as the final destination, and an additional behaviour called a buffer, which can store temporarily messages received from the producer until the consumer is ready to receive them. Obviously, the problem of producer and consumer could be solved by using a synchronous system without the need for buffering, but it would unnecessarily slow down the whole system to the slowest component, thus resulting in long times between consecutive ticks. By isolating the producer and consumer with their own clocks and separating them with another buffer clock domain with its own clock, the system enables smooth operation of producing and consuming the messages by introducing intermediate message buffering. The roles of three CDs are:

1. Producer (PABRO): This CD generates a data and sends it to a Buffer CD when both signals A1 and B1 are captured from the environment. In this example, the data is the Fibonacci sequence generated within the producer.
2. Buffer: This CD receives the Fibonacci sequence from PABRO via a channel and queue the received numbers into a First-In-First-Out (FIFO) buffer. The data is fetched and transferred to a Consumer CD via a channel when the Consumer is ready to receive in order they are received from the PABRO.

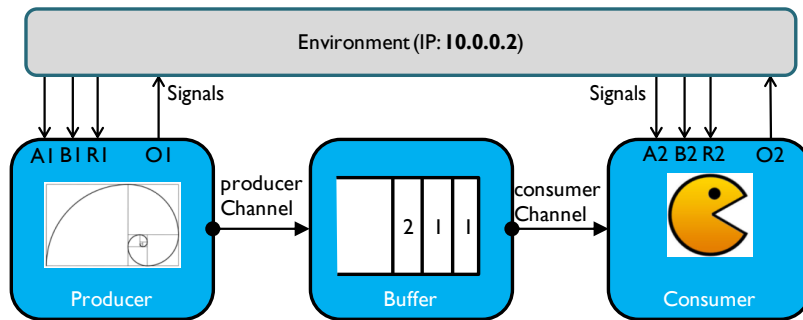


Figure 3.8 A graphical illustration of PCABRO

3. Consumer (CABRO): This CD receives the Fibonacci sequence that is queued in the Buffer CD. Like PABRO, CABRO is only able to fetch the numbers from Buffer when both signals A2 and B2 are captured from the environment.

A graphical illustration of the PCABRO example is shown in Figure 3.8. The control logic of both the producer and the consumer CD are based on the ABRO example from Section 3.3. Here, we declare a ABRO reaction, which is shown at lines 4-12. All the Java classes used in this program are also imported as shown at lines 1-2. The ABRO reaction is instantiated in both PABRO (line 15) and CABRO (line 29) that wait for the presence of signals A and B. Once these signals are captured, signal O will be emitted from the ABRO, which triggers generation of the next Fibonacci sequence in case for PCABRO. The number will be sent to the Buffer CD via a channel from PABRO, which is shown at line 22. On the other hand, CABRO will initiate a receiving operation with the Buffer immediately upon emission of signal O (lines 34-36). The Buffer CD consists of three concurrent reactions where the one of them manages the Java FIFO object (lines 63-81) and others exchange data with PABRO and CABRO CDs (lines 45-61), respectively. These reactions communicate with each other using internal signals, which are declared at lines 43-44. The first reaction (lines 45-51) checks the presence of `bFull`, which indicates that the queue is full. If the queue is not full (i.e. `!bFull` is true) the reaction tries to receive the next Fibonacci sequence from PABRO via `fromPABRO`. A signal `insert` is then emitted with the received value (`#fromPABRO`) that inserts the value into the FIFO queue. The second reaction (lines 53-61) checks if the queue is not empty (i.e. `!bEmpty` is true) and fetches the last element from the queue by emitting `get`. The value of the fetched element will be emitted as a value of signal `q` which is captured and sent to CABRO at lines 58-59. The third reaction (lines 63-81) implements the FIFO queue using a Java library class called `FIFO`. The size of the queue is 10, which is initialised at the beginning of the reaction (line 64). As explained in Section 2.5, the first two reactions cannot share the FIFO Java object created in the third reaction because all variables created in a reaction are local, i.e. only accessible in the reaction where they are created. Instead, reactions interact with each other using signals that manipulate the FIFO queue.

The PCABRO example demonstrates how GALS approach decouples a program into several synchronous islands, which can run with different clock speeds. The GALS model is well suited for designing complex systems that consist of multiple parts operating simultaneously for achieving multiple goals and are run by different speeds, which may be particularly important for the systems that are distributed in nature and run on different computers.

```

1  import systemj.util.FIFO;
2  import systemj.util.Fibonacci;
3
4  reaction ABRO(:input signal A, input signal B, input signal R, output signal O) {
5      loop {
6          abort(R){
7              {await(A);} || {await(B);}
8              emit O;
9              halt;
10         }
11     }
12 }
13
14 PABRO(input signal A1,B1,R1; output signal O1; output int channel fromPABRO;)->{
15     ABRO(:A1,B1,R1,O1)
16     ||
17     {
18         Fibonacci fib = new Fibonacci();
19         loop {
20             abort(R){
21                 await(O1);
22                 send fromPABRO(fib.next());
23             }
24         }
25     }
26 }
27
28 CABRO(input signal A2,B2,R2; output signal O2; input int channel toCABRO;)->{
29     ABRO(:A2,B2,R2,O2)
30     ||
31     {
32         loop {
33             abort(R){
34                 await(O2);
35                 receive toCABRO;
36                 System.out.println("Received Fibonacci number : " + #toCABRO);
37             }
38         }
39     }
40 }
41
42 Buffer(input int channel fromPABRO; output int channel toCABRO;)->{
43     int signal insert, q;
44     signal bFull, bEmpty, get;
45     {
46         loop {
47             await(!bFull);
48             receive fromPABRO;
49             emit insert(#fromPABRO);
50         }
51     }
52     ||
53     {
54         pause;
55         loop {
56             await(!bEmpty);
57             emit get;
58             await(q);
59             send toCABRO(#q);
60         }
61     }
62     ||
63     {
64         FIFO b = new FIFO(10);
65         loop {
66             if(b.isFull())
67                 emit bFull;
68             else if (b.isEmpty())
69                 emit bEmpty;
70
71             abort {
72                 {await(bFull); sustain bFull;} || {await(bEmpty); sustain bEmpty;}
73             } case (get && insert) do {
74                 emit q(#insert);
75             } case (!bEmpty && get) do {
76                 emit q(b.dequeue());
77             } case (!bFull && insert) do {
78                 b.enqueue(#insert);
79             }
80         }
81     }
82 }

```

Figure 3.9 PCABRO example.

Chapter 4 SystemJ Execution Models and Their Implementation

In this chapter we discuss the execution model of SystemJ program, the dependence of the model on the platform (architecture) on which it executes, as well as the role of the run-time system that supports the execution of the program on micro and macro level. Here, we consider execution of each CD and its constituent parts micro level, while the execution of multiple CDs, which includes their scheduling, is considered macro level.

4.1 SystemJ Execution Models

In this section we describe some key options within the SystemJ execution model. The key aspects of the model are related to the translation of the source code of the SystemJ program into an intermediate representation called Asynchronous Graph Code (AGRC). The AGRC representation is the key for linking the program to the execution platform and consequent generation of the object code. We have two major pathways from AGRC to the execution platform

1. The AGRC graph will be compiled into Java code, which requires only Java (Virtual) Machine (J(V)M) with Java bytecodes as the low level executable instructions for its execution or
2. The AGRC graph is mapped onto two instruction sets executed by two different machines, one set specifically designed to support efficient code for concurrency and reactivity (control code) of SystemJ and the other one that requires Java (Virtual) Machine with bytecodes as the instruction set to support parts of SystemJ program that directly use Java.

For the first approach the minimum we need is any general purpose processor platform that has Java Virtual machine (JVM) implemented or the platform that is a processor which is Java Machine (JM), with bytecodes as its native instructions. The second option requires execution of control code on separate machine that can be virtual (implemented on any instruction set) or physical machine with custom tailored instruction set. Both pathways allow execution of SystemJ programs with some subtle differences that will be explained briefly in the next sections. The options of execution models will be introduced and explained on an example of a concrete program, software system, from industrial automation domain.

4.1.1 Asynchronous Graph Code - AGRC

As an example of SystemJ program and its execution model we describe a bottle capping device (the Capper) of an *Automated Bottling Machine* (ABM) [11], which is presented with full details in Chapter 6. The Capper SystemJ controller is described as a clock domain, which is one of the five clock domains that cooperatively control the ABM. The Capper source code is shown in Figure 4.1 (a). The clock domain has a set of input and output signals for establishing the status of and actuating the Capper, which is responsible for sealing a bottle filled with liquid using a plastic cap. The control logic for such operation comprises several SystemJ reactions as shown in Figure 4.1(a) on lines 21-61. For example, to seal a bottle, the Capper has to perform three operations concurrently: (1) clamp a bottle while being sealed with a plastic screw cap (reaction denoted as R211, lines 28-37), (2) tightly grip the cap (R212, lines 39-42) and (3) twist the gripper to seal the bottle (R213, lines 44-53). Furthermore, there is an

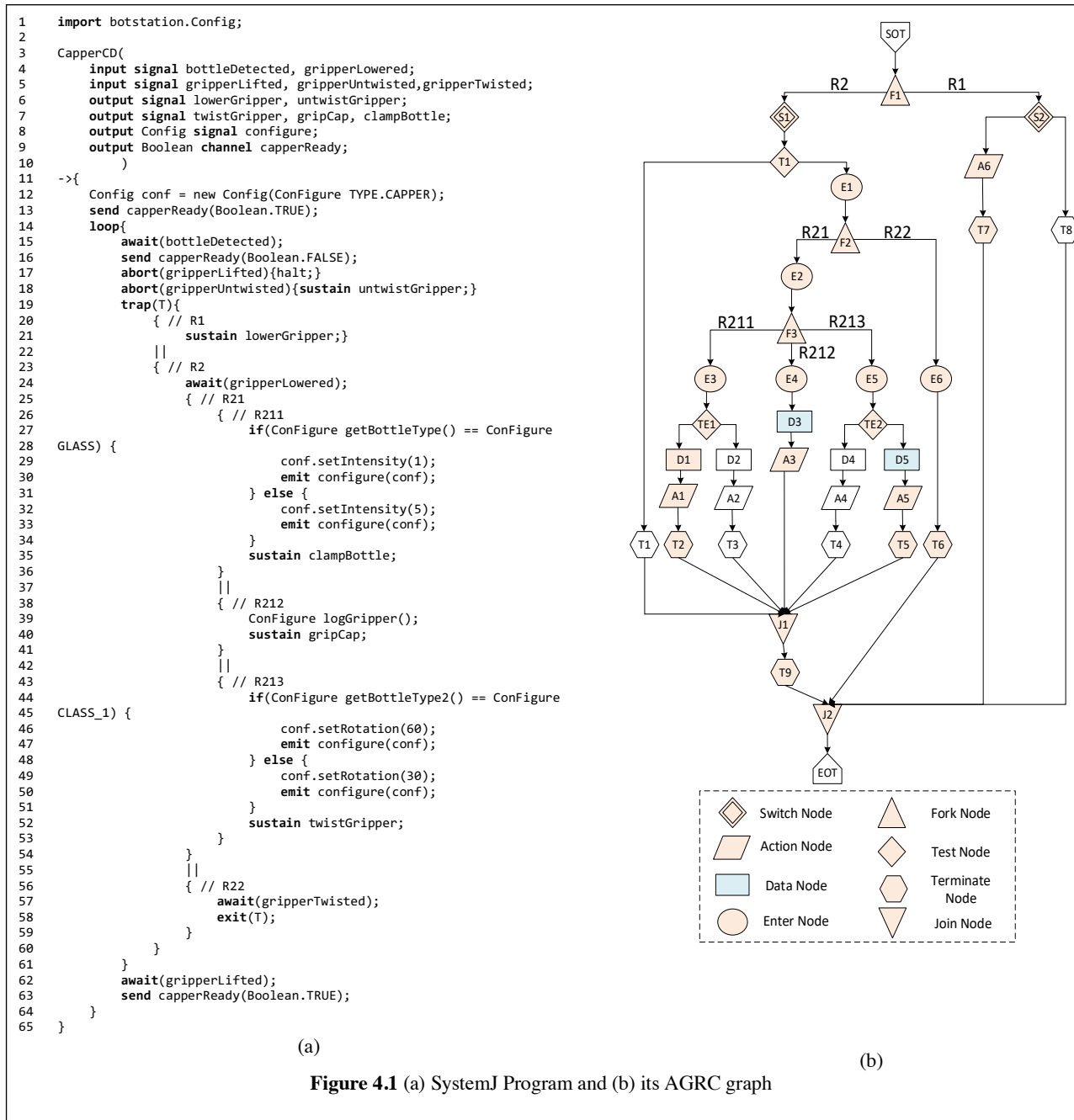
additional reaction R22, which monitors a completion of the capping operation, which runs in parallel with reactions R211, R212 and R213 as shown on lines 56-59. These reactions continuously emit output signals to actuate the mechatronic devices, parts of the Capper, to perform the aforementioned operations. Sustaining the signal `clampBottle` on line 36 clamps a bottle until it is pre-empted by reaction R22 on line 58, which changes the control-flow to jump out of the enclosed trap body on lines 21 and 61. Before emitting the output signal, R211 checks the type of the bottle (e.g. glass) and configures clamping intensity using the Java object called `conf`. R212, on the other hand, logs a status of the gripper (line 40) before the capping operation. Lastly, R213 checks the type of the cap and sets the angle of the gripper accordingly (lines 45-51). The setting is configured via emitting the Java object `conf` to the environment (lines 31, 34, 46 and 49). All reactions operate synchronously in lock-step with the logical clock of the Capper clock domain, and execute one clock tick of computation before exchange of the information via broadcasting the signals and their values and continuing with the next tick.

The AGRC, which consists of two types of schedulable units, called control-nodes and data-nodes, generated from the Capper clock domain is shown in Figure 4.1(b). Note that the figure only shows a subgraph of the whole graph due to space limitations. Execution of all reactions is delimited by two nodes called SOT (start-of-tick) and EOT (end-of-tick). These two nodes also separate subgraphs of the AGRC that represent different clock domains. Execution model of SystemJ assumes traversal of nodes from SOT to EOT according to the control flow. All input signals that belong to the clock-domain are captured at SOT and any output signals emitted during the execution are transmitted to the external environment at EOT. Depending on the evaluation of conditional constructs in the program, control-flow traverses different paths in the AGRC. In this example, we consider one such path, which is denoted as coloured nodes in Figure 4.1(b). All SystemJ kernel statements are translated to one or more control-nodes, which are marked green in the figure. At the same time, all Java statements between any two SystemJ statements are encapsulated into a single data-node, shown as blue in the figure.

4.1.2 Execution Model for JVM platforms

The execution of computations for one logical tick starts within the SOT node. At each fork node the parent reaction suspends and spawns two or more child parallel synchronous reactions, which all have to complete computations that belong to one logical tick before eventually join and return control to the parent reaction. The global requirements on the execution of the active reactions (those that are not suspended) is that in each logical tick of the clock domain all active reactions must complete computations corresponding to one logical tick. The order in which reactions execute is defined by the execution schedule and the result of execution does not depend on the schedule. The only requirement on the execution of the nodes that belong to a reaction is that they are executed control dependencies of the nodes in the reaction. All reactions may have Java computations carried out on the objects declared within the reaction that are local for the reaction. This means that there are no data dependencies between reactions. All Java objects are collected and reinitialised in the next logical tick of the clock domain. If it is necessary to retain the values of computation stored in these objects beyond tick boundary, SystemJ requires the use of signals which, when emitted, up-date their values and retain them until the next emission. When all reactions complete a tick, then the signal and channel values are up-dated and the execution of the clock domain can continue with the next logical tick. Up-date of the signal values, which include all external

signals, is called housekeeping and carried out before the clock domain starts the execution of the next logical tick. Housekeeping is performed within the help of run-time system.



The fact that system does not allow data dependencies between reactions, by isolating the scope of Java variables and objects to a reaction, brings two benefits to the execution model: (1) forked reactions can be executed in parallel if the execution platform allows it, which is possible in multi-core systems, and (2) execution of the nodes of AGRC in different reactions can interleave in any order, as long as the control dependences of the nodes within each reaction are preserved. These two facts are used to generate different specific schedules of not only reactions but also the nodes and achieve specific targets, e.g. worst case reaction times, where the reaction time is the time to execute a logical tick of the clock domain, using static schedules, which is of interest for hard real-time systems developed in SystemJ.

4.1.3 Execution Model for Specialised Time-Predictable Platforms

Two examples of the schedules of SystemJ program from Figure 4.2(a) demonstrate the implementation of the SystemJ execution model and how the AGRC nodes are scheduled on *Network on Chip-Heterogeneous Multi-Processor* (NoC-HMP) [12] by showing an execution trace of the AGRC nodes on a configuration with a single ReCOP and a single JOP cores of the NoC-HMP. ReCOP and JOP have different instruction sets tailored for the efficient execution of control code and data operations, respectively. The figure shows the trace from the execution of node F2, since the execution of the predecessor nodes does not influence the overall clock-domain reaction time. Furthermore, we omitted the communication times of the interconnect network for the sake of simplicity. Execution of a SystemJ clock domain always starts on ReCOP. Whenever ReCOP encounters a data node, e.g. D1, it asks the JOP to execute the corresponding Java code (method) by launching a *data call*. Upon the completion of the method, the JOP returns a result of the execution to the ReCOP, which can continue its execution from the next AGRC node. On a configuration from Figure 4.2.(a), the trace shows a relatively long reaction time, which is the time to complete one tick of each reaction in a clock domain, because all the data-nodes are executed sequentially on a single JOP. On the other hand, with an additional JOP processor, multiple data calls can be executed in parallel. As a result, the overall reaction time of the clock-domain has been shortened as shown in Figure 4.1(b). This example shows that the execution of a SystemJ program can be accelerated by employing more JOP cores, in which case a suitable schedule that guarantees certain worst case reaction time, also called *Guaranteed Reaction Time* (GRT) should be found. Finding the GRTs is the main task of the *Design Space Exploration* (DSE) [13] and is out of scope of this document.

4.2 SystemJ Execution Platforms

Up until now, we have described SystemJ systems using the computational entities, reactions and CDs, and communication objects, signals, and channels. When creating a system, we need to compose those entities in more organised and meaningful way so that they can co-operate with each other to perform complex tasks. Furthermore, as a SystemJ program can run on various platforms, there are platform-specific properties and dependencies that need to be considered, such as IO and runtime environment, when deploying CDs. In this section, we first give an overview of currently available execution platforms for SystemJ programs. The remaining part of the chapter will be devoted to the description of the SystemJ runtime environment for those platforms.

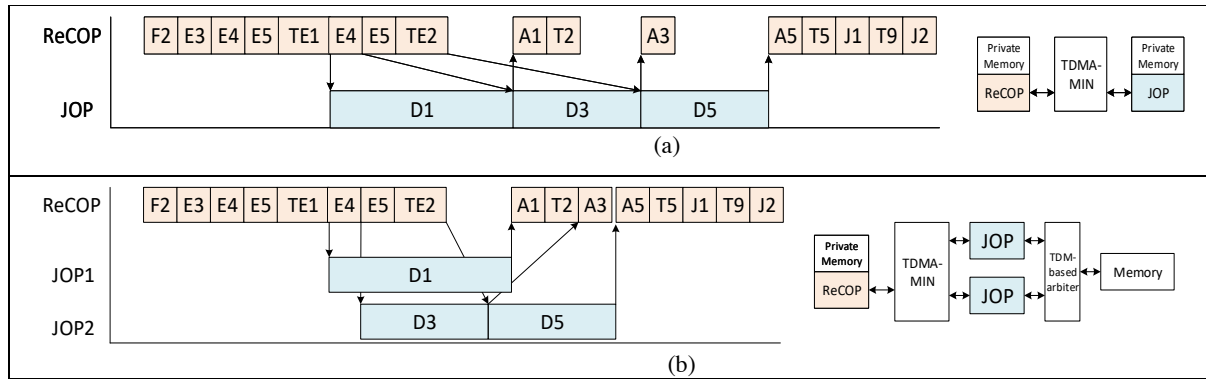


Figure 4.2 SystemJ Program Schedules for different multi-core configurations (a) and (b)

4.2.1 Java Virtual Machine (JVM) Based Platforms

The most commonly used execution platform for SystemJ programs is a PC. SystemJ designers can easily debug and test their programs by executing SystemJ code compiled and executed on any Java Virtual Machine (JVM) that supports Connected Limited Device Configuration (CLDC) 1.1 specification or higher. A SystemJ program that targets JVM is first compiled into a collection of Java source code files, where each file (.java) corresponds to a CD in the SystemJ program. Those CDs are compiled using `javac` and deployed along with the configuration file that initialises the program by parsing a configuration file and extracting necessary information to run. In case when the target platform is resource-constrained, i.e. cannot parse the configuration file, the SystemJ toolchain pre-process the configuration file and statically configures the program before deployment. One example of such resource-constrained platform is Java Optimized Processor (JOP) [14].

4.2.2 Java Optimized Processor (JOP) and the Multi-Core Platforms

The research has been done in recent years to run SystemJ programs efficiently on JOP or different multi-core architectures that use JOP as a building block core and described in multiple works. In these works, JOP is extended with additional capabilities so that it can execute some of the SystemJ-specific instructions directly such as signal emissions and forking reactions. The first attempt was introduced in [15] where the authors added a set of special Java bytecodes to JOP. These bytecodes are translated into a sequence of micro-code instructions that perform reactive operations in the SystemJ programs. The author's following work [16] enhanced JOP further with the support of more SystemJ-specific bytecodes that executes control flow part of the SystemJ program more efficiently. The overhead due to context-switching between the control and data (Java) part of the SystemJ program is greatly reduced in [17]. Lastly, the multi-core processor based execution platform for the SystemJ programs is introduced in [18].

The multi-core processor architecture presented in [18] is homogeneous platform, i.e. it consists of identical processors. There is also heterogeneous multi-core platform [12] that consists of two types of time-predictable cores. In [12], authors present a RISC-type core called ReCOP for executing control flow and JOP for executing algorithmic part of the SystemJ program. Designers can instantiate any numbers of ReCOPs and JOPs and they are interconnected with a time-predictable TDMA-based network-on-chip (NoC) called TDMA-NoC. Since ReCOP, JOP and TDMA-NoC are time-predictable, the SystemJ program deployed to this heterogeneous platform is also time-analysable. Extensive researches have been done in the context of timing analysis of SystemJ programs [13], [19], [20], all related to the use of SystemJ in hard real-time systems.

4.3 Developing SystemJ Programs

This section addresses the development of SystemJ programs for standard JVM-based platforms used in majority of SystemJ programs.

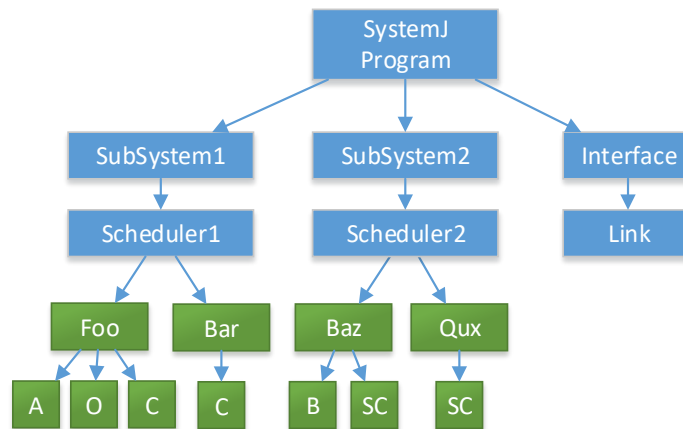
4.3.1 Program Components and Organisation

The SystemJ program/system complies with the GALS model of computation where the program is decomposed hierarchically into a single layer of mutually asynchronous CDs at the top level of hierarchy and then each CD can be decomposed into any number of hierarchical layers of mutually synchronous reactions with complex parent-children relationship (behavioural hierarchy). While the program specification is a logic description of the designed system that describes fully system's functionality, decomposition into smaller design units and their interactions, it does not assume anything about the execution platform on which it will be deployed. As designed, system can be immediately executed on the development computer, effectively providing a powerful validation platform. However, from the design and execution platform point of view we can perform additional groupings of CDs according to the strategy of deployment of clock domains on the target platform (physical machines and JVMs) and create another abstraction and representation of the SystemJ program.

SystemJ programs are compiled into Java code by default and can be deployed to practically any platforms that support JVM, with or without operating system. When running the SystemJ program, however, designers would need additional support from the runtime system so that they can utilise platform-specific functionalities, such as, for example, the details of the interaction with the environment via IO ports and interfaces or using different types of networks and network protocols. Although, SystemJ currently supports many widely-used platforms, designers can also customise the SystemJ runtime to incorporate new platforms and specific input and output capabilities. This section introduces an overview of the SystemJ runtime system and how it can be customised for new platforms.

```
Foo(input String signal A; output String signal O; output String channel C2;)->{
  /* Foo's code */
}
Bar(input int channel C1;)->{
  /* Bar's code */
}
Baz(output signal B; input int channel C1;)->{
  /* Baz's code */
}
Qux(output String channel C2;)->{
  /* Qux's code */
}
```

(a)



(b)

Figure 4.3 The runtime system's view of the SystemJ program consisting of four CDs

With respect to aforementioned selection of the execution platform, we can compile and group CDs into the subsystems so that these groups create another layer of system organisation that reflects the deployment of CDs and their scheduling. This also takes into account communication mechanisms used to implement channels within the program, as well as the underlying mechanisms of implementation of interface (external) input and output signals and outlines the factors that have to be taken into account in SystemJ run-time system, which has to create conditions for all abstract entities and objects used in a program.

A SystemJ program, represented at the top of hierarchy tree, consists of a number of subsystems that group CDs that are executed on the same JVM or Java processor governed by a separate scheduler and information that describes the interface and communication mechanisms used to connect machines on which subsystems are deployed. Each subsystem has its own run-time support that maps subsystem's input and output signals and channels by which the subsystem's CDs communicate with the CDs in other subsystems onto physical inputs/outputs and communication links that exist between computers on which subsystems run.

For example, Figure 4.3 illustrates the SystemJ program consisting of four CDs. As shown in Figure 4.3(a), CDs Foo, Bar, Baz, and Qux have their own input and output signal and channel ports for exchanging data with the environment and between each other. Figure 4.3(b) shows the runtime components for this example program that manage execution, communication, and runtime resources for the CDs.

1. *SystemJ Program/System*: It represents the highest-level abstraction that describes the interface to the system's external environment. At the same time it is an entry point of the program that initialises the designed system's components and run-time system. It also describes the mappings of abstract signals and channels on concrete communication protocols and physical interfaces and manages exchange of data with environment and between subsystems.
2. *Subsystem*: SystemJ program is further divided into one or more subsystems where each subsystem manages execution of the CDs which belong to the subsystem. Each subsystem has its own JVM on which it runs, as well as its own run-time support. Subsystems can be, for example, run on the same processor and different JVMs or they can be distributed across multiple execution platforms/machines.
3. *Scheduler*: It is used to manage execution of CDs within a subsystem, where different subsystems can have different schedulers. For example, in Figure 4.3(b), Scheduler1 for SubSystem1 manages CDs Foo and Bar whereas Scheduler2 for SubSystem2 manages CDs Baz and Qux. Scheduler decides how CDs are executed. Unless specified differently, by default a scheduler runs all CDs it manages cyclically on per CDs' tick basis.
4. *Link*: It is a physical connection that enables communication and exchange of data between CDs in different subsystems. It allows the implementation of channels used to communicate between those CDs. For example, when there is a channel connection between Bar and Baz through a channel C1 there must be a link established between the corresponding subsystems to exchange data between the CDs. If two subsystems run on the same processor, e.g. on different cores, the link is typically implemented in shared memory accessible to be used by those channels. Similarly, the link can be implemented by using operating system's sockets for message exchange. If the subsystems are on different networked processors/machines, then typically standard communication protocols are used to implement links (e.g. TCP/IP, UDP, industrial

network protocols). Obviously, this is dependent on the way how the processors or processor cores are interconnected and for non-standard communication mechanisms customised links have to be implemented as a part of SystemJ run-time support.

5. *Interface*: It manages instances of links and resolves the location of CDs in the network. For example, an interface manager is able to find an IP address of the machine that runs SubSystem2 in the network where the CD Baz is executed.

Exact organisation of each SystemJ program, deployment of its parts and mapping to the available and used resources is defined in a program/system configuration in a special dedicated file. As this file can be rather complex for large programs and execution platform, it can be split into smaller configuration files that are associated with the individual subsystems and contain only those configuration aspects that affect individual subsystems. The subsystem configuration is parsed by the subsystem run-time support before the subsystem starts its operation and becomes alive and part of the system to which it belongs.

4.3.2 Specifying a Configuration File

As one can see in Figure 4.3, building a system requires additional information that describes the components interconnections and create hierarchical relationship with each other. We use XML to specify the configuration file that describes a complete system and the SystemJ runtime then uses this file to initialise and start running a whole or part of the system. We are going to use the example shown in Figure 4.3 to build a system with the following specification:

1. CDs Foo and Bar run on a machine with an IP address 10.1.1.100 and a port 5000.
2. CDs Baz and Qux run on a machine with an IP address 10.1.1.200 and a port 5000.
3. Foo captures signal A on port 1000.
4. Foo receives data on channel C2 from Qux.
5. Bar receives data on channel C1 from Baz.
6. Both Foo and Baz emit signals O and B to environment via IP address 10.1.1.50 and port 2000.

An overview of this system is illustrated in Figure 4.4. On the top-level of the program hierarchy, we have a SystemJ program. In a configuration file, we specify this as follows:

```
<System xmlns="http://systemjtechnology.com">
</System>
```

Here, the element System is declared with an attribute xmlns, which is a namespace for this configuration file. Declaring this namespace is important as the SystemJ runtime checks validity of the configuration file using an XML schema. Now we need to declare two subsystems with global IP addresses:

```
<System xmlns="http://systemjtechnology.com">
  <SubSystem Name="SubSystem1">
  </SubSystem>
  <SubSystem Name="SubSystem2">
  </SubSystem>
</System>
```

We provide a name of each subsystem using the attribute Name. Each subsystem consists of two CDs thus we specify them as children for each SubSystem:

```
<System xmlns="http://systemjtechnology.com">
  <SubSystem Name="SubSystem1">
    <ClockDomain Name="FooCD" Class="Foo"></ClockDomain>
    <ClockDomain Name="BarCD" Class="Bar"></ClockDomain>
  </SubSystem>
  <SubSystem Name="SubSystem2">
    <ClockDomain Name="BazCD" Class="Baz"></ClockDomain>
    <ClockDomain Name="QuxCD" Class="Qux"></ClockDomain>
  </SubSystem>
</System>
```

In this file, each clock domain's name is appended with an identifier "CD". Each CD's compiled Java class file is specified using the `Class` attribute value. Note that the names of the classes should be fully qualified, e.g. `java.util.ArrayList`, if they are in any arbitrary `syspackage`. In SystemJ, signal is exchanged with environment typically over a certain type of communication protocol. In order to emit signal, the runtime system first transforms a SystemJ signal object into a packet, which is recognised by the communication protocol established with the environment. This packet is then transmitted to the environment at the end of CD tick, during housekeeping phase. In this example, we are going to use Java's built-in object serialiser that transforms the SystemJ output signal object into a stream of bytes. The object is then transmitted to the environment via a TCP socket. It is also assumed that the environment provides input signals to CDs via the same mechanism. Although designers can implement their own versions of the TCP-based signal exchange mechanism, the SystemJ runtime system already provides a library that enables TCP communication using signals:

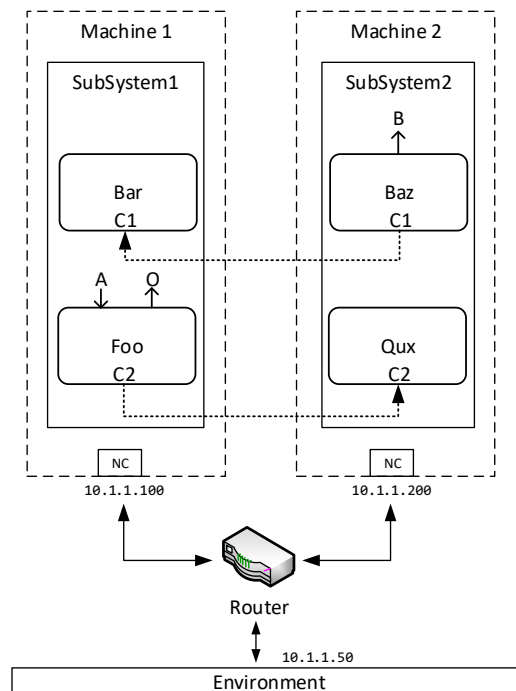


Figure 4.4 Distributed System configuration

```

<System xmlns="http://systemjtechnology.com">
  <SubSystem Name="SubSystem1">
    <ClockDomain Name="FooCD" Class="Foo">
      <iSignal Name="A" IP="10.1.1.100" Port="1001"
Class="com.systemj.ipc.TCPReceiver"/>
      <oSignal Name="O" IP="10.1.1.50" Port="2000"
Class="com.systemj.ipc.TCPSender"/>
    </ClockDomain>
    <ClockDomain Name="BarCD" Class="Bar"></ClockDomain>
  </SubSystem>
  <SubSystem Name="SubSystem2">
    <ClockDomain Name="BazCD" Class="Baz">
      <oSignal Name="B" IP="10.1.1.50" Port="2000"
Class="com.systemj.ipc.TCPSender"/>
    </ClockDomain>
    <ClockDomain Name="QuxCD" Class="Qux"></ClockDomain>
  </SubSystem>
</System>

```

Input and output signals are specified using elements `iSignal` and `oSignal`, respectively. Designer should provide attribute `Class` for each signal that specifies SystemJ inter-process communication (IPC) class. For example, here we used `com.systemj.ipc.TCPSender` for the output signals `O` and `B`, and `com.systemj.ipc.TCPReceiver` for the input signal `A`.

Connections between input and output channel ports of two CDs are established by specifying the name of a CD and the name of the channel ports as shown in Figure 4.. Input and output channel connections for each CD are defined by `iChannel` and `oChannel`, respectively. For `iChannel`, a connection to the output channel port is specified by the attribute `From`, for example `From="BazCD.C1"` for the input channel `C1` of `FooCD` where the syntax of the value is `<CDName>.<Output channel port name>`. On the other hand, the attribute `To` is used for specifying a connection for `iChannel`, for example `To="QuxCD.C2"` for the output channel `C2` of `FooCD`. Since subsystems that manage their CDs are deployed to different machines, we also need to specify the location of those machines in the configuration file. The locations of two machines for `SubSystem1` and `SubSystem2` are defined by the `Links`. `Links` are implementing communication protocols that are used for exchanging data, e.g. channels, between subsystems. For example, we specified in the configuration file that both subsystems can talk to each other through an IP address specified under `Links`. `Links` are formed by specifying one or more interface types used by the machines. In the example, we are using a simple TCP/IP network interface for both machines. When any CD tries to communicate with another CDs, which runs on a remote machine, through channels, the SystemJ runtime resolves the location of the remote CD via this configuration file. In case when all CDs run on a same machine and in the same subsystem, we can omit the `Interconnection` element in the configuration file that will instruct the SystemJ runtime to establish channel connections using a shared memory in the JVM heap.

The configuration file shown in Figure 4. gives a complete specification of the system where all the information including constituent CDs and their mappings on the physical and logical (JVM) machines, and I/O specifications are defined. This configuration file is called a *Global Configuration File (GCF)*. Each subsystem requires its own configuration file because each of them runs on their own execution platform. To run a subsystem, a designer needs to mark it in the GCF with an attribute `Local`. For example, the `SubSystem2` with `Local` value `true` in Figure 4. indicates that this configuration file is for running the `SubSystem2`. The configuration file with a subsystem set with `Local="true"` is called a *Local Configuration*

File (LCF). In our example, we need two LCFs since there are two subsystems to run. A complete list of the elements and their attributes that are used in the LCF is shown in 0.

4.4 Runtime System and its Customisation

SystemJ designers normally configure their systems via configuration files. Very often, TCP is the primary choice of communication protocol for implementing channels and signals and it is already included in the SystemJ release. However, since SystemJ targets a wide range of execution platforms with different IO and network capabilities, the SystemJ runtime system allows designers to introduce new communication protocols that can be used to implement channels and signals. In Section 4.3, we have shown how to write a GCF where signals and channels are implemented using TCP protocol using the Inter-Process Communication (IPC) library. In this section, we are going to show how to implement a SystemJ IPC class that implements signal communication via a file.

In order to introduce a new input or output signal IPC class, a designer needs to create a Java class inheriting from one of the following abstract classes:

1. `com.systemj.ipc.GenericSignalSender` for output signals.
2. `com.systemj.ipc.GenericSignalReceiver` for input signals.

Figure 4.5 visualises these classes in a UML diagram. `GenericSignalReceiver` class has two abstract methods that need to be implemented by its subclasses, and two concrete methods, which can be overridden if needed:

- `public abstract void configure (Hashtable data)` – This method is called only once during the initialisation phase of the SystemJ program. The parameter `data` contains a set <Key,Value>, which maps attribute name (Key) to its value (Value) in a configuration file.
- `public void getBuffer(Object[] obj)` – This method is called for every clock-domain *End of Tick* (EOT). The runtime expects a new status and a new value of

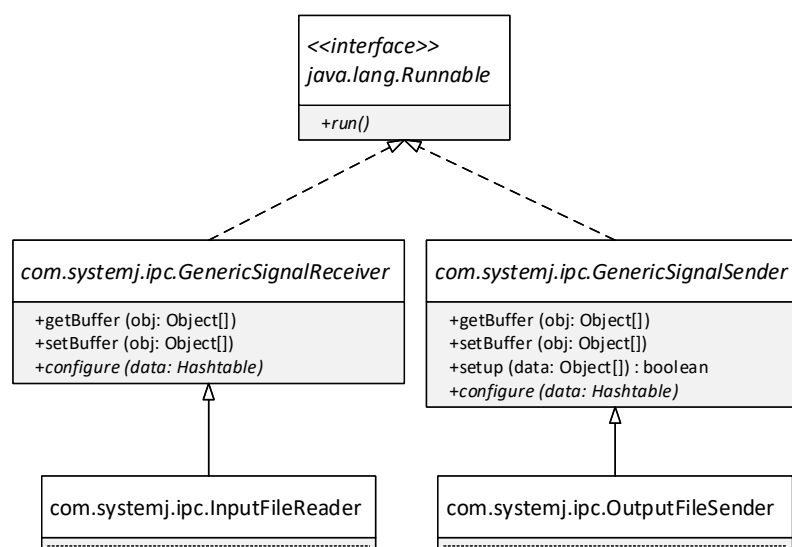


Figure 4.5 An UML diagram of the IPC abstract classes

the corresponding signal to be stored in `obj[0]`. (`java.lang.Boolean`) and `obj[1]` (`java.lang.Object`), respectively. This method can be overridden if required.

- `public void setBuffer(Object[] obj)` –The default implementation passes a signal status (`obj[0]`) and a value (`obj[1]`) to the current instance of the signal class. This is can be overridden if required.
- `public abstract void run()` – During the initialization phase, the runtime invokes `GenericSignalReceiver.start()` to spawn a thread that runs `GenericSignalReceiver.run()`. Spawning a new thread may be required if `GenericSignalReceiver.run()` consists of blocking operations (e.g. `ServerSocket.accept()`). Otherwise, leave this an empty method.

```
<System xmlns="http://systemjtechnology.com">
  <Interconnection>
    <Link Type="Destination">
      <Interface SubSystem="SubSystem1"
Class="com.systemj.ipc.TCPIPInterface" Args="10.1.1.100:100/>
      <Interface SubSystem="SubSystem2"
Class="com.systemj.ipc.TCPIPInterface" Args="10.1.1.200:100/>
    </Link>
  </Interconnection>
  <SubSystem Name="SubSystem1">
    <ClockDomain Name="FooCD" Class="Foo">
      <iSignal Name="A" IP="10.1.1.100" Port="1001"
Class="com.systemj.ipc.TCPReceiver"/>
      <oSignal Name="O" IP="10.1.1.50" Port="2000"
Class="com.systemj.ipc.TCPSender"/>
      <oChannel Name="C2" To="QuxCD.C2"/>
    </ClockDomain>
    <ClockDomain Name="BarCD" Class="Bar">
      <iChannel Name="C1" From="BazCD.C1"/>
    </ClockDomain>
  </SubSystem>
  <SubSystem Name="SubSystem2" Local="true">
    <ClockDomain Name="BazCD" Class="Baz">
      <oSignal Name="B" IP="10.1.1.50" Port="2000"
Class="com.systemj.ipc.TCPSender"/>
      <oChannel Name="C1" To="BarCD.C1"/>
    </ClockDomain>
    <ClockDomain Name="QuxCD" Class="Qux">
      <iChannel Name="C2" From="QuxCD.C2"/>
    </ClockDomain>
  </SubSystem>
</System>
```

Figure 4.6 A Global Configuration File (GCF)

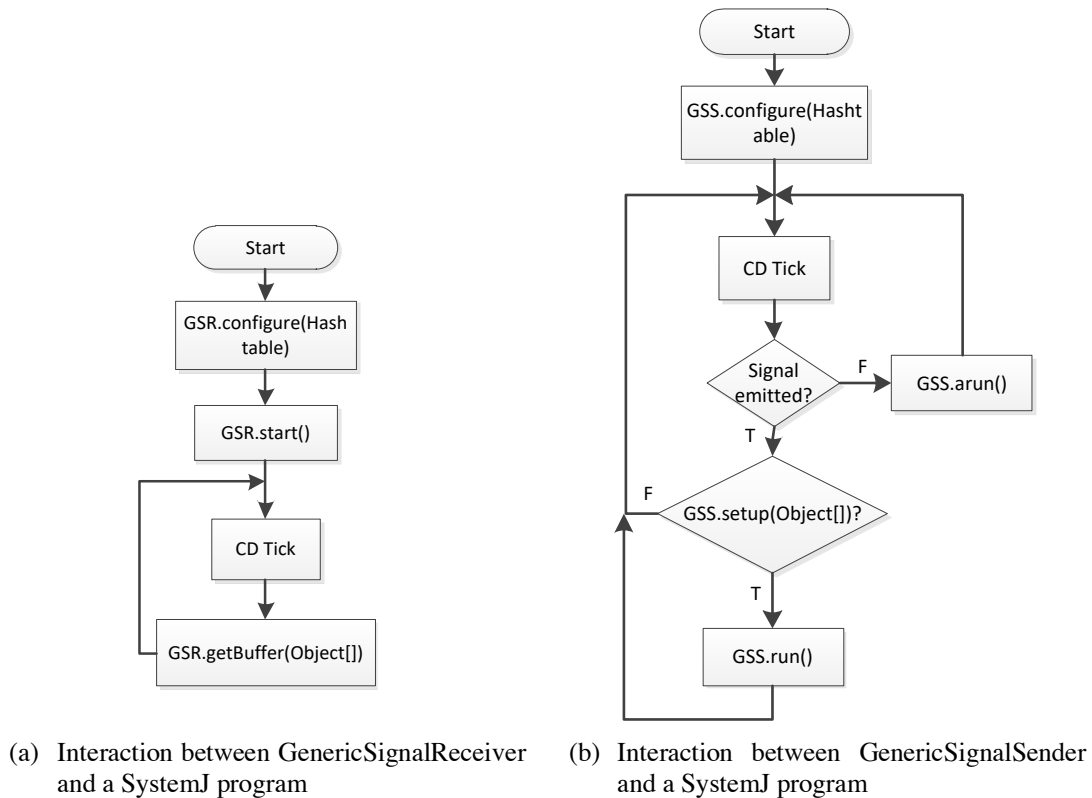


Figure 4.7 Execution flow of the GenericSignalReceiver and GenericSignalSender classes

On the other hand, GenericSignalSender class has the following methods:

- `public abstract void configure(Hashtable data)` – See *GenericSignalReceiver*
- `public boolean setup(Object[] data)` throws `RuntimeException` – This method is called when the signal is emitted in the previous tick, and should return either true or false (see Figure 4.b). `data[0]` always contains `java.lang.Boolean.TRUE` and `data[1]` contains a signal value (if there is any), which is any Java object. This method can be overridden if required.
- `public abstract void run()` – This method is called for every clock-domain End Of Tick (EOT) only if previously called `setup(Object[] data)` returned true.
- `public void arun()` – This method is called when the signal is *not* emitted (absent) in the previous tick. This method is empty by default and can be overridden if required.

Both subclasses of the signal sender and receiver invoke their methods according to the execution flow shown in Figure 4.7. For example, an instance of GenericSignalReceiver (GSR) runs `configure()` upon start of the program to initialise input signals with the attributes specified in a configuration file. GSR then invokes `start()` that starts a thread for listening on inputs from the environment. Note that this thread does not immediately update a status and a value for the signal, instead it is done in the `getBuffer()` method after a CD tick completion. Similar to GSR, instances of GenericSignalSender (GSS) run `configure()` method for initialising output signals. GSS checks if any signals are emitted during the last CD

tick. If emitted, the corresponding signal's `setup()` followed by `run()` are invoked, otherwise `arun()` is invoked (see Figure 4.). In the next section, we are going to show examples of IPC classes that extend GSR and GSS.

4.4.1 InputFileReader extending GenericSignalReceiver

We can use files to provide inputs to the SystemJ CDs. This is useful when IO is mapped to files (e.g. Linux) or to quickly check the functionalities of a CD by providing input events via a text file. When a designer wants to bind an input signal A to a file named `input.txt`, it needs to be configured in the LCF as follows:

```
<iSignal name="A" Class="com.systemj.ipc.InputFileReader"
File="input.txt"/>
```

Each line in `input.txt` is then read as a value for the signal A where numbers of lines in the file correspond to a CD tick count. One can implement the method `configure` for the `InputFileReader` as follows:

```
@Override
public void configure(Hashtable data) throws RuntimeException{
    filename = (String)data.get("File");
    try{
        br = new BufferedReader(new FileReader(filename));
    }catch(FileNotFoundException e){e.printStackTrace();}
}
```

The `getBuffer` method reads the file line by line for each CD tick, and reopens the file when it reaches End-Of-File. Note that the signal is considered absent (i.e. `obj[0]=false`) when the line consists of a character `';`' otherwise present (i.e. `obj[0]=true`). `obj[1]` is assigned with a signal value, which is read from `input.txt`. The concrete implementation of `getBuffer` is shown below:

```
@Override
public void getBuffer(Object[] obj){
    try{
        String line = br.readLine();
        if(line == null){
            br.close();
            br = new BufferedReader(new FileReader(filename));
            line = br.readLine();
        }
        if(line.trim().equals(";")){
            obj[0] = Boolean.FALSE;
        }
        else{
            obj[0] = Boolean.TRUE;
            obj[1] = line;
        }
    }catch(Exception e){e.printStackTrace();}
}
```

Since reading the input file can be done for each time `getBuffer` is called, there is no need to keep the thread alive. In this case, we can leave the `run` method empty:


```
@Override
public void run() { }
```

Lastly, we need to call the super class' (GenericSignalReceiver) inside the constructor of the InputFileReader:

```
public InputFileReader(){
    super(); // Initialises the buffer
}
```

Calling super will initialise a buffer for the input signal that temporarily holds a status and a value before they get retrieved by the CD.

4.4.2 OutputFileWriter extending GenericSignalSender

When designers want to write an emitted signal to a file, they can map the output signal to OutputFileWriter as follows:

```
<osignal Name="O" Class="com.systemj.ipc.OutputFileWriter"
File="output.txt"/>
```

A CD that emits O with a type `java.lang.String` will then write the signal's value to a file `output.txt`. Similar to `InputFileReader`, the method configure for `OutputFileWriter` opens a file with the name specified in the configuration file:

```
@Override
public void configure(Hashtable data) throws RuntimeException {
    if(data.containsKey("Name")){
        signalName = (String)data.get("Name");
    } else throw new RuntimeException("The configuration parameter
'Name' is required!");

    if(data.containsKey("File")){
        dir = (String) data.get("File");
    } else throw new RuntimeException("The configuration parameter
'Path' is
required!");

    try{
        String fileName = new String(dir);
        file = new File(fileName);
        file.createNewFile();
    }catch(IOException ioe){ throw new RuntimeException("Cannot create a
file"); }
}
```

Note that the file is created when it cannot be found in the specified location. We do not have to override setup method as we are going to use the default method in `GenericSignalSender`. However, we still need to call the parent class' constructor in the `OutputFileWriter's` constructor:

```

public OutputFileWriter(){
    super();
}

```

Finally, the emitted signal is written to the file in the run method with a signal value (String), which can be retrieved from `GenericSignalSender.buffer[]`. The concrete implementation is shown below:

```

@Override
public void run() {
    Object[] obj = super.buffer;
    String data = (String) obj[1];

    try{
        writer = new BufferedWriter(new FileWriter(file,true));
        writer.write(data,0,data.length());
        writer.write("\n", 0, 1);
        writer.flush();
        writer.close();
    } catch(IOException e){
        e.printStackTrace();
    }
}

```

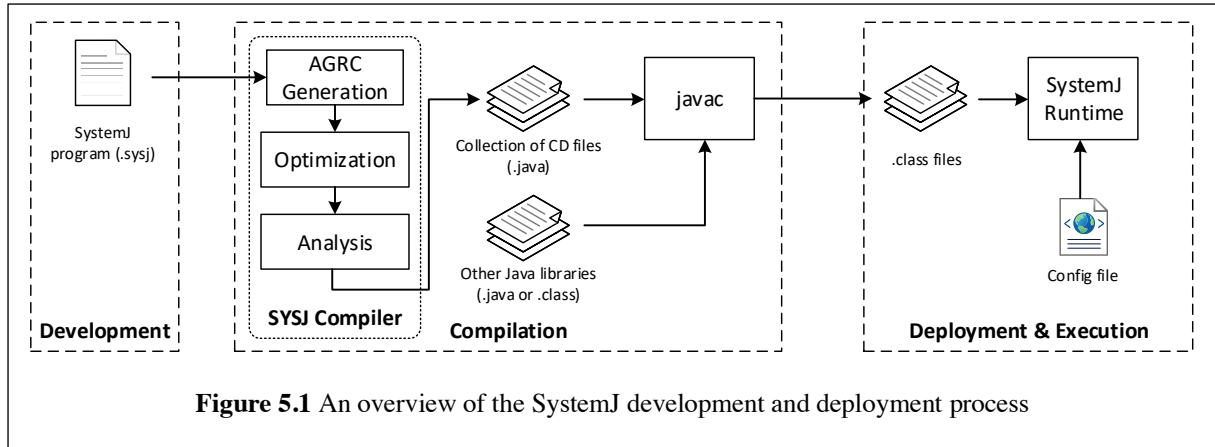
Chapter 5 Distribution and Deployment of Software Systems

This chapter presents compilation strategies of SystemJ programs and describe how the back-end code is generated from the high-level description. A development guidance for compiling and running a SystemJ program is presented next. SystemJ programs can interface with programs written in other languages. We introduce some of the options that are available to achieve this and present future directions to build heterogeneous software systems where other software technologies can co-exist with SystemJ technology. Finally, we present some of the features that are recently introduced, which will be a part of the SystemJ development framework in the near future.

5.1 SystemJ Program Development and Deployment Strategies

A default target of the SystemJ program is a platform with a JVM. In that case, the SystemJ compiler is basically a source-to-source compiler that generates a Java source code from a SystemJ program. The generated Java code is then compiled into a bytecode using `javac` which is deployed to the target any JVM-enabled platforms. The overall compilation flow is shown in **Error! Reference source not found.** Development environment for SystemJ programs can be any text editor. Since the basic syntax colouring and line indentation rules of the SystemJ language are similar to Java, and most text editors support Java and provide comprehensive features for the language, SystemJ developers often choose their favourite editor for SystemJ program development. Alternatively, programmers can opt for the Atom editor (<https://atom.io/>) and install SystemJ development plugins that extends Atom with integrated development environment for the SystemJ language. The source code of the plugin is available at <https://github.com/arrayoutofbounds/sysj>.

A file extension of the SystemJ source code is `.sysj`. The source code is given as an input to the SystemJ compiler where it generates *Asynchronous GRaph Code* (AGRC) intermediate representation, the foundation of the GALS formal model of computation as well as back-end code generation. AGRC is further optimised by the SystemJ compiler where the compiler removes any unreachable paths that are trivial to find and reduce the size of the graph. Optionally, the AGRC is further analysed through several processes during which functional and real-time properties are checked [13], [19], [21]. The output of the SystemJ compiler is a collection of Java source files (`.java`), where each file corresponds to a CD in the SystemJ source file. These Java source codes along with any Java libraries used in the SystemJ program are compiled by the Java compiler that generates the class files (`.class`) to be deployed to the target platform. To run the program, one needs the generated CD class files, a configuration file (`.xml`), and the SystemJ runtime support (RTS). Typically, these files are copied to the target platform by the developer to start execution of the SystemJ program.



are compiled by the Java compiler that generates the class files (.class) to be deployed to the target platform. To run the program, one needs the generated CD class files, a configuration file (.xml), and the SystemJ runtime support (RTS). Typically, these files are copied to the target platform by the developer to start execution of the SystemJ program.

5.2 Asynchronous Graph Code (AGRC)

AGRC is a graph that consists of primitive nodes of certain types that build control-flow of the SystemJ program. Figure 5.2 shows a simple AGRC structure that is generated from the SystemJ code below:

```
CD1(input signal A;)->{
    await(A);
    System.out.println("A");
}
CD2(input signal B;)->{
    present(B) { System.out.println("B"); }
}
```

The program consists of two CDs that each prints the name of the input signal they receive. AGRC begins with a root node called *Asynchronous Fork Node* (AFN) whose children (outgoing edges) indicate CDs. Since there are two CDs in the example code, the corresponding AFN has only two children as shown in Figure 5.2. Control-flow of a SystemJ program traverses from the AFN to the branches of CDs until it reaches *Asynchronous Join Node* (AJN) that indicates the corresponding CD's end-of-tick. Whenever control-flow encounters any AGRC node during the traversal, it triggers a sequence of actions on the executing platforms according to the semantics of the corresponding node. Based on the node type, control-flow then branches to one of the children or further spawns one or more concurrent reactions simultaneously. For example, when control-flow encounters the *Switch Node* (S1) shown in Figure 5.2, it branches to one of the node's three outgoing edges based on the encoding, which represents states of the CD. This encoding (state) is updated by the *Enter Nodes* (e.g. E1) depicted as ellipses in the graph. For example, when control-flow enters the branch 1 of S1, the enter node E1 encodes S1 to 2, which, in effect, makes control-flow to enter the branch 2 of S2 in the next CD tick. The *Terminate Nodes* (e.g. TN1) depicted as diamonds encode statuses of concurrent entities in a CD, for example reactions, to determine whether to continue or stop the execution of the spawned entities. The *Test Node* (e.g. T1) tests the presence of signal(s) and they are created by signal checking statements in a program such as `await`, `abort`, and etc. Signal emissions and Java statements in the CDs are shown as *Action Nodes* (rectangular shaped node, e.g. A) and *Data Nodes* (parallelogram shaped nodes, e.g. "Emit A" and "Emit B") in the AGRC, respectively. SystemJ reactions are spawned by the *Fork Node* (e.g. F1) and their termination statuses generated by the terminate nodes are tested in the *Join Node* (e.g. J1).

The SystemJ compiler translates the AGRC into Java code, which can be compiled by the Java compiler into .class files. These class files, along with the SystemJ runtime and a configuration file, are deployed and executed on any JVM-enabled platforms. For example, see Figure 5.3, which is Java code generated from the CD1 in Figure 5.2. The runtime classes for the SystemJ objects, such as CDs, signals, and channels, are shown at lines 1 to 5. The states

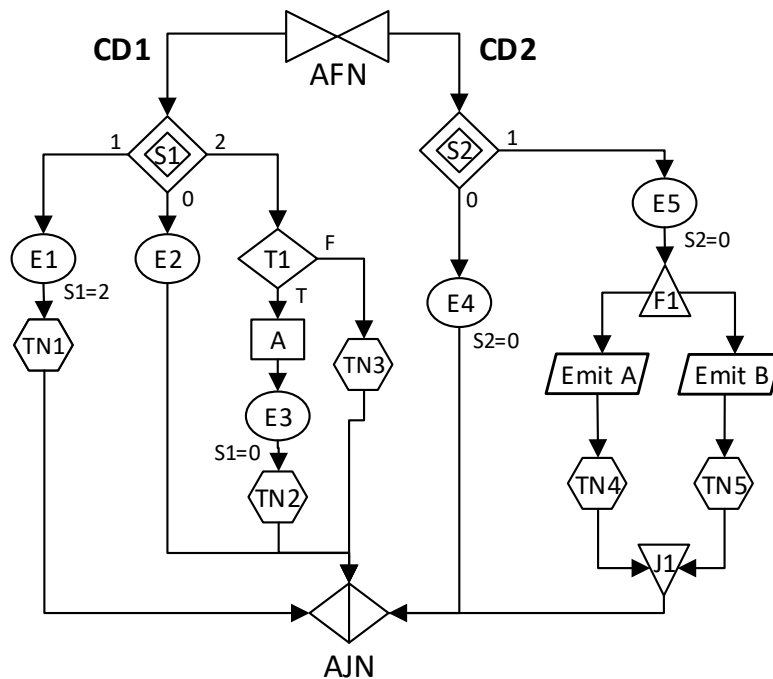


Figure 5.2 An example of AGRC intermediate representation

```

1      import java.util.*;
2      import com.systemj.ClockDomain;
3      import com.systemj.Signal;
4      import com.systemj.input_Channel;
5      import com.systemj.output_Channel;
6
7      public class CD1 extends ClockDomain {
8          public Signal A = new Signal("A", Signal.INPUT);
9          private int S1 = 1;
10
11         public void runClockDomain() { /* Omitted code for a CD tick */ }
12
13         public void run() {
14             /* Simplified code */
15             A.gethook(); // Getting signal A from the environment
16             runClockDomain(); // Running the CD tick
17             A.updateSignal(); // Updating signal status and value with the previous tick
18         }
19     }

```

Figure 5.3 Java source code generated from the AGRC.

of the CD, such as signal A (line 8) and S1 (line 9), are declared as the class fields. The runtime executes a tick of the CD1 by invoking the method `runClockDomain()` (line 11). The simplified housekeeping code, which updates signal values and channels, is shown at lines 13-18. The SystemJ runtime manages a life-cycle of this CD and the communication between the environment and other CDs through concrete protocols (e.g. networks) at higher-level, which are not shown in this figure for brevity.

5.3 Compiling and Running SystemJ Programs on Development Computers

The SystemJ Development Kit (SJDk) is freely available on <https://github.com/hjparker/systemj-release>. The archive file named `sjdk-<version>-<SHA1>.tgz` consists of two directories.

1. `bin` – Contains executable scripts for the compiler and the runtime.
2. `lib` – Contains JAR files for the tools.

When the path to the `bin` directory is added to the `PATH` environment variable, programmers can use two commands, `sysjc` and `sysjr`, which are for compiling and running the SystemJ programs, respectively. For example, consider you have the SystemJ ABRO program in the current directory as shown below:

```

$ ls
ABRO.sysj ABRO.xml

$ _

```

Then the `ABRO.sysj` can be compiled with the following command:

```

$ sysjc ABRO.sysj
$ ls
ABRO.class ABRO.java ABRO.sysj ABRO.xml

$ _

```

To run ABRO, `sysjr` is used the local configuration file as its input:

```
$ sysjr ABRO.xml
```

ABRO has only one CD that is executed on a single subsystem. However, a SystemJ system can be created using multiple subsystems. In this case, one needs to create a local configuration file for each subsystem. Creating LCFs for different subsystems are merely copying the global configuration file with a small modification as shown in Figure 5.4. The figure shows LCFs for two subsystems, which are derived from the example shown previously in Figure 4.5 in Chapter 4. As one can see, we simply added an attribute “Local” with a Boolean value “true” to the subsystem to be executed on the target platform, without changing the rest of the configuration file. To execute subsystems, one needs to copy the corresponding LCF (.xml), CD class files (.class) and the SystemJ runtime (JAR files in the lib directory, and possibly the `sysjr` script file) to each target platform. Depending on the selected platforms, the deploying files may need to be included in vendor-specific toolchains, which are not the scope of this manual.

5.4 Development Tools

The SystemJ development kit provides a number of tools that help designers to debug and integrate CDs with other tools and software systems.

```
<System xmlns="http://systemjtechnology.com">
  <Interconnection>
    <Link Type="Destination">
      <Interface SubSystem="SubSystem1"
Class="com.systemj.ipc.TCPIPInterface" Args="10.1.1.100:100/>
      <Interface SubSystem="SubSystem2"
Class="com.systemj.ipc.TCPIPInterface" Args="10.1.1.200:100/>
    </Link>
  </Interconnection>
  <SubSystem Name="SubSystem1" Local="true">
    <!-- Omitted brevity -->
  </SubSystem>
  <SubSystem Name="SubSystem2">
    <!-- Omitted brevity -->
  </SubSystem>
</System>
```

(a)

```
<System xmlns="http://systemjtechnology.com">
  <Interconnection>
    <Link Type="Destination">
      <Interface SubSystem="SubSystem1"
Class="com.systemj.ipc.TCPIPInterface" Args="10.1.1.100:100/>
      <Interface SubSystem="SubSystem2"
Class="com.systemj.ipc.TCPIPInterface" Args="10.1.1.200:100/>
    </Link>
  </Interconnection>
  <SubSystem Name="SubSystem1">
    <!-- Omitted brevity -->
  </SubSystem>
  <SubSystem Name="SubSystem2" Local="true">
    <!-- Omitted brevity -->
  </SubSystem>
</System>
```

(b)

Figure 5.4 Local configurations files for (a) subsystem 1 and (b) subsystem 2

5.4.1 The SystemJ Environment Simulator

A SystemJ CD is a reactive program whose execution behaviour is heavily influenced by the input stimuli from the external environment. When developing SystemJ programs, designers would need to write a test bench that simulates those inputs which can be quite a burden in some cases. For this scenario, the SystemJ tool provides the environment simulator, which can generate input signals to CDs under test. In order to use the simulator, one needs to remove the corresponding signal entries in the local configuration file as shown below.

```
<System xmlns="http://systemjtechnology.com">
  <SubSystem Name="ABRO_SS" Local="true">
    <ClockDomain Name="ABRO" Class="ABRO">
      <!-- In this case all the input signals will be controlled -->
      <!-- Signal 0 can also be removed if it is not actually received by
the env. -->
    </ClockDomain>
  </SubSystem>
</System>
```

In this example, all the input and output signals for the ABRO CD will be controlled by the environment simulator. When running the program, one needs to give an option `-s2` to `sysjr`.

```
$ ls
ABRO.class ABRO.java ABRO.sysj ABRO.xml

$ sysjr -s2 ABRO.xml
```

The command will start the ABRO with a GUI panel, which can be used to generate signals A, B and R. The execution of the CD is controlled by the button, e.g. “Run ABRO” in Figure 5.5, which executes the corresponding CD for one tick. The layout of the simulator is shown in Figure 5.5. Designers can set status of signals by clicking the tick boxes on the panel, and optionally values in either `java.lang.Integer` or `java.lang.String` object.

5.4.2 The sysjnetapi Library

A SystemJ CD can exchange information with other programs via input and output signals, which is usually done via TCP sockets. In such a case, developers need to manage TCP servers

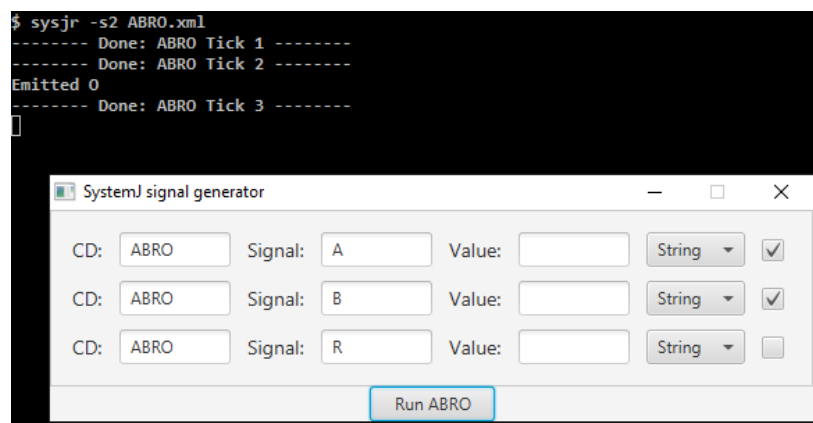


Figure 5.5 Debugging ABRO via the environment simulator.

(most likely involve multithreading) and created sockets on their own. Moreover, they need to understand data-structure of the SystemJ signals and how to initiate data exchange with the SystemJ runtime system, which can be quite a burden. The sysjnetapi library abstracts away such delicate details, which can introduce additional complexities to the program. The library is written in Java but it can be easily migrated to other languages, too. The library, as well as its documentation, is freely available at <https://github.com/hjparker/sysjnetapi>.

5.4.3 Pre-processing Local Configuration File

It is often the case that some target platforms, especially embedded environment, only support minimal standard libraries and a small program memory, which make parsing of the XML format of the LCF during runtime unsuitable. In this case, designers can pre-process LCFs and generates an entry point of the subsystem before deployment. For example, one can generate Main.java from the LCF (Main.xml) via the `-x` option.

```
$ ls
CD.java CD.class Main.xml MyCD.sysj

$ sysjc MyCD.sysj
$ sysjr -x Main.xml
$ ls
CD.java CD.class Main.java Main.sysj Main.xml MyCD.sysj
```

The file Main.java contains the Java static main method which is the entry point for the program Main.sysj given that the contents of the LCF is well-formed and correct. Main.java can be compiled using `javac` with the SystemJ runtime jar files (Section 5.3) added to the classpath.

5.4.4 Signal interfaces

In addition to the method described in Section 4.4, for introducing new runtime classes for input and output signals, designers can instead write a series of Java methods in a single class. Consider the following simple SystemJ CD.

```

CD1(input String signal A; output int signal B;)->{
    int i;
    loop {
        await(A);
        emit B(i++);
    }
}

```

One way to interface A and B with the external environment using a protocol that is not in the SystemJ runtime system is to implement new Java classes which extend `GenericSignalReceiver` and `GenericSignalSender` as explained in Section 4.4. Alternatively, designers can introduce a collection of Java methods in a single class that handles communication with the external environment as shown below.

```

package com.systemjx.ipc;

public class SignalInterface {

    public static Object[] CD1_A_I(){
        // Returning Object[] consists of [0] a signal status and [1] a
        signal value
        return new Object[] { true, "Foo" };
    }

    public static void CD1_B_O(Object[] s){
        // s[0] contains the emitted signal status and s[1] contains the
        value
    }
}

```

For the input signal A, one can create a Java static method called `CD_A_I()` that returns an object array consists of signal status and its value stored in the first and the second index, respectively. For the output signal B, the static method `CD_B_O(Object[] s)` can be created where the signal's status and value can be retrieved in the object `s`, which is an argument for the method. Those are all the methods of the Java class `com.systemjx.ipc.SignalInterface`, which is loaded dynamically by passing an option `-s` for `sysjr` as shown below:

```
$ sysjr -s LCF.xml
```

The naming rules for the input and output signals in the `SignalInterface` class are as follows:

1. For the input signals – public static Object[]
 <CDName>_<SignalName>_I()
2. For the output signals – public static void
 <CDName>_<SignalName>_O(Object[])

To execute these methods in the `SignalInterface` class for the input and the output signals, designers need to remove the corresponding element in the local configuration file. The following is the contents of the `LCF.xml` for the example program shown previously.

```

<System xmlns="http://systemjtechnology.com">
  <SubSystem Name="SS1" Local="true">
    <ClockDomain Name="CD" Class="CD">
      <!-- Empty I/O specification for the CD. -->
      <!-- In this case, the signals are exchanged in
com.systemjx.ipc.SignalInterface -->
    </ClockDomain>
  </SubSystem>
</System>

```

5.4.5 Gradle project

SystemJ programs can be built and executed using the Gradle build tool (<https://gradle.org>), which is an open source build automation tool commonly used for building Java programs. A template SystemJ Gradle project can be downloaded from <https://github.com/hjparker/systemj-gradle-project>. The compilation of the SystemJ program is configured via the Gradle task called `SystemJ`, which is a type of `com.systemj.CompileSystemJ`. For example, see the following code, which is in `build.gradle` file.

```

task SystemJ(type: com.systemj.CompileSystemJ) {
    // Libraries needed to compile SystemJ programs
    classpath buildscript.configurations.classpath

    // OPTIONAL: Adding options explicitly, value: keys.
    //           Give an empty string for the non-valued options (e.g. --
silence).
    // options "--config-gen": "default.xml", "--silence": ""

    // OPTIONAL: Adding directories containing .sysj files, default:
src/main/systemj
    //           sysj files are recursively searched and compiled
    // target files('path-to-dir')

    dependsOn compileJava
}

```

Here, we added all the dependencies for compiling a SystemJ program by calling the `classpath` method. Optionally, designers can pass other compiler options by using `options` method whose argument is a map of compiler options, e.g. `<option>: <value>`. By default, all the `.sysj` files in the `src/main/systemj` directory are compiled recursively. Designers can also optionally add other directories containing the `.sysj` files using the method `target`.

Running SystemJ program is done via the built-in Gradle task `JavaExec` as shown below.

```

task runApp(type: JavaExec) {
    // Adding required Java classpaths
    classpath 'build/classes/main', buildscript.configurations.classpath

    // This is the entry point of the SystemJ runtime to run the program.
    main = 'com.systemj.SystemJRunner'

    // A configuration file is loaded from
src/main/configurations/default.xml
    args 'src/main/configurations/default.xml'
}

```

The runApp task adds the SystemJ runtime jar files as well as compiled CDs to the classpath and run the SystemJ program with the local configuration file default.xml. Note that the entry point of the SystemJ runtime is `com.systemj.SystemJRunner`. To build and run the example program given in the template project, run `./gradlew` (for Linux) or `gradlew.bat` (for Windows) at the top-level project directory.

5.5 Experimental Features

This section gives a description of newly introduced features in the SystemJ tool that are still under development.

SystemJ subsystems can be deployed to target execution platforms via network using the following tools:

1. `sysjd` that deploys SystemJ subsystems to target machines
2. `sysjs` that starts a server for receiving an incoming subsystem

Using these tools, designers can start or replace SystemJ subsystems dynamically. For examples the following command start a SystemJ server on machine 1 that receives an incoming subsystem on a port 1000.

```
# On a machine 1 with IP address 123.456.789.10
$ sysjs -v 0.0.0.0:1000
Jun 12, 2017 11:53:33 AM com.systemj.dist.StartServer main
INFO: Created a server listening on server://0.0.0.0:1000
```

Now one can transmit a subsystem (named SS1) using `sysjd`.

```
# On a machine 2
$ sysjc HelloWorld.sysj # first compile the SystemJ program that prints
Hello World
$ sysjd -j --deploy SS1://123.456.789.10:1000 LCF.xml
```

The `sysjd` command will parse the contents of the local configuration file in `LCF.xml` and transmit all the CDs that are under the subsystem tagged as `Local="true"`. Once the subsystem is received, machine 1 will start running it as shown below.

```
$ sysjs -v 0.0.0.0:1000
Jun 12, 2017 11:53:33 AM com.systemj.dist.StartServer main
INFO: Created a server listening on server://0.0.0.0:1000
Jun 12, 2017 11:53:37 AM com.systemj.dist.Server configure
INFO: SubSystem received, creating an instance ..

Hello World
```

If we can change `HelloWorld.sysj` to print “Hello NZ” and re-run `sysjd`, machine 1 will restart the subsystem using the newly received subsystem:

```
$ sysjs -v 0.0.0.0:1000
Jun 12, 2017 11:53:33 AM com.systemj.dist.StartServer main
INFO: Created a server listening on server://0.0.0.0:1000
Jun 12, 2017 11:53:37 AM com.systemj.dist.Server configure
INFO: SubSystem received, creating an instance ..

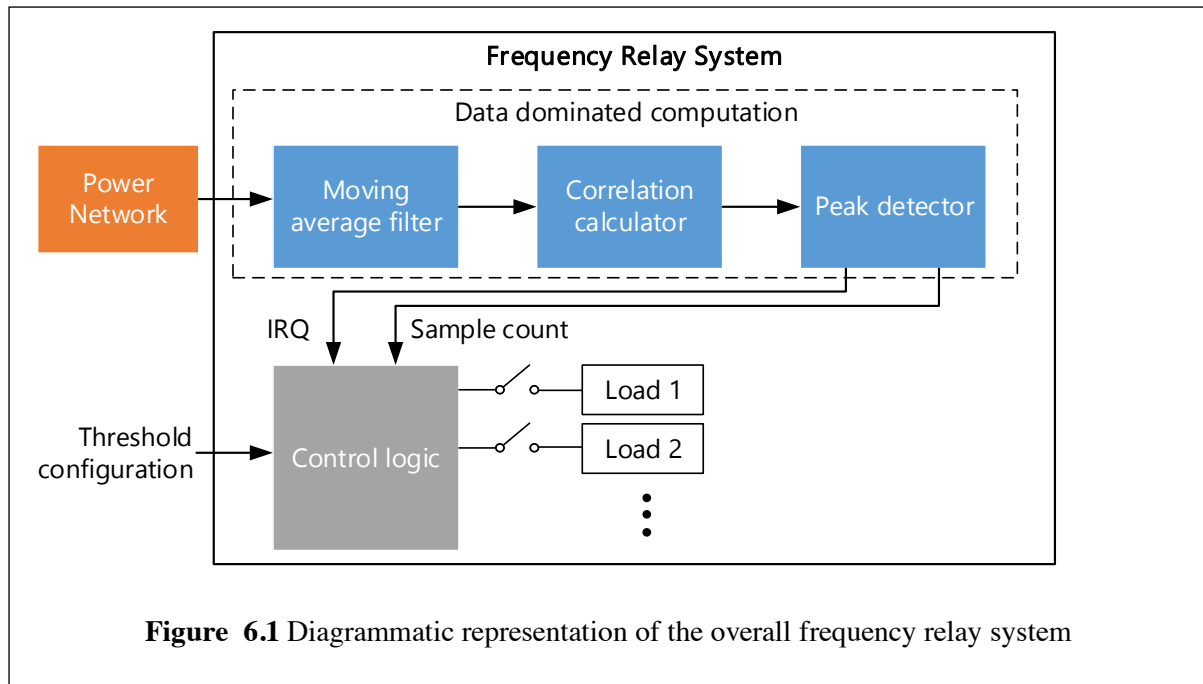
Hello World
Jun 12, 2017 11:55:32 AM com.systemj.dist.Server configure
INFO: SubSystem received, creating an instance ..
Hello NZ
```

Chapter 6 Comprehensive Case Studies

In this chapter, we present two SystemJ case studies that illustrate the use of the language in the designs of more complex real-life systems. The first example is the frequency relay, which measures the frequency of a power system that operates at 50 Hz and helps regulating the frequency if it is below certain threshold or changes too fast by shedding the loads. An example of an interoperable industrial automation and robotics system is presented next, where we demonstrate a design of controllers for the bottling system that fills the bottles with liquid and then closes them with the caps before delivery as the final product. The system employs a Baxter robot to carry out some auxiliary workpiece manipulation tasks.

6.1 Frequency Relay

A frequency relay is a low-cost system that interfaces directly with the power network to measure current frequency and rate of change of frequency in real-time and then control the overall load by switching off or on individual devices/appliances using triacs (Triode for Alternating Current) connected to a fuse box that supply power to the household's sun-circuits. The detection of the abnormal power network frequency and maintaining a stable network frequency is crucial to avoid possible disasters of the power system. Frequency relay is a hard real-time system, which requires to measure frequency and its rate of change and react on changes if they are outside the give thresholds within a firm deadline. The algorithms to measure frequency and its rate of change are of digital signal processing type, computationally intensive and data-driven and typically require implementation of at least some of their parts in digital hardware. However, software approaches can also be employed if the target execution platform, typically high-performance processor, always meets the real-time requirements of the frequency relay. Control-dominated parts of algorithms employed in frequency relay, which typically do relatively simple calculations and make decisions for load balancing, on the other hand, can be always efficiently implemented in software. In this section, we are going to present an implementation of the frequency measurement part of the frequency relay in SystemJ, where the data-dominated computations are described in Java as an integral part of SystemJ. An overview of the frequency relay system is illustrated in Fig. 6.1., where we clearly separate (1) data-dominated functionality that takes power signal samples as the inputs and generates counts of higher frequency clock (in this case sampling frequency clock) between the two points that represent a period of the input waveform (blue shaded blocks) and (2) control-dominated functionality that calculates the frequency and its rate of change and controls the loads by switching them on and off (grey shaded block). While the main input into the relay is power signal, the only other input into the relay comes from outside and represents the thresholds of frequency and rate of change at which the relay has to switch off (shed) the loads (these are the parameters that can be changed by a higher-level control system).



The data-dominated part of the frequency relay system has as its input samples of power system signal taken at high-enough sampling frequency and consists of three parts connected into a cascade and that execute continuously:

1. *Moving average filter*. In a real system, power signal that enters the frequency relay is disturbed by noise that creates random fluctuations of otherwise ideal sinusoidal signal. The raw power signal samples are filtered by a moving average filter, generates the average values of fixed number of samples and moves in time to the next sequence of samples by removing oldest and adding the next incoming sample into the window. The size of the window is a design parameter. The role of the filter is to remove some of the noise present in the raw data samples and enable detection of points of periodicity of input signal.
2. *Correlation calculator* computes auto-correlation values of the filtered signal at any points of time that indicates symmetrical property of the waveform. For example, the correlation value is larger where it is more symmetrical about that data being evaluated and reaches maximum at the so-called reference points. Similar to the moving average filter, a designer can parameterise the window size (range) used for calculating correlation values.
3. *Peak detector* uses the correlation values received to detect a distance between the maximal and the minimal point of the input power signal at any given time. The output of the peak detector block is a signal, which indicates the detection of a peak, and a sample count, which is a number of samples between the current and the previous peak. As the samples are taken at fixed sampling frequency (and period), the number of samples between two peaks is directly proportional to the time between two peaks and the period of the power signal.

Based on the sample count, the control logic can find frequency of the current input signal, and based on the threshold configuration, it can react quickly by disconnecting or reconnecting loads to maintain a stable network frequency. The graphical overview of the SystemJ CD that implements the data-dominated part of the frequency relay is shown in Figure 6.2. Since the system is required to react on the changes in the input frequency in real-time, it is implemented as a single (synchronous) CD to avoid potential communication delays in case when the sub-functionalities are distributed across CDs. There are three inputs to the frequency relay: input samples (SampleIn), moving average window (MAW) size, and symmetry window (SW) Size. The average moving filter removes noise in the input power signal samples by averaging sample values within the MAW. The result is then transferred to the correlation calculator via signal AveResult and the auto-correlation about the sample at k within the SW is evaluated using the following equation.

$$C = \sum_{n=0}^{\frac{SW}{2}} f(k+n-1)f(k-n) \quad (1)$$

The computed correlation value is passed to the peak detector where it identifies the maximum correlation value and emits a result to the external environment via signal SampleOut whose value is an integer number. This value indicates a number of samples since the previously detected maximum correlation value. As we know the sampling rate of the input wave, which is by default 16KHz, the frequency of the signal can be calculated using the following equation:

$$f_{signal} = \frac{f_{sample}}{n_{ADC}} \quad (2)$$

where f_{signal} is the frequency of the input signal, f_{sample} is the sampling rate, and n_{ADC} is the number of samples between the two peaks identified by the peak detector block.

To simulate the frequency relay implemented in a SystemJ CD, we have developed a web-based front-end that generates input samples to the CD and also displays results. The overall software architecture of our system is illustrated in Figure 6.3. In this system, the SystemJ frequency relay CD is a service to the user that interacts with the web-based frontend. Users provide four inputs; an input wave formula in continuous time t , which describes a series of

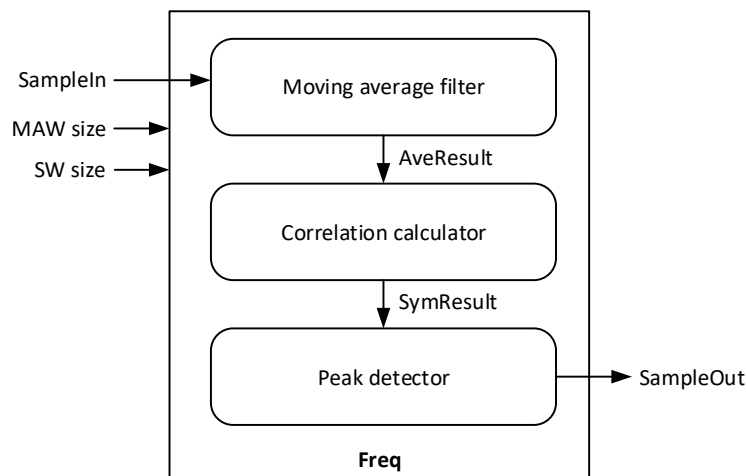


Figure 6.2 The SystemJ frequency analysis CD

sinusoidal waves including harmonics, sampling frequency, symmetry window size, and moving average window size. Upon submitting these inputs, the wave samples are generated using the given formula on the client's (user) machine and sent to the webserver via an HTTP post request. The server application then receives the wave samples along with MAW and SW sizes, which are passed as inputs to the SystemJ frequency relay program. Outputs of each blocks in the frequency relay are returned to the server application via standard streams and as a HTTP response to the client's web browser, where the results are displayed as line graphs.

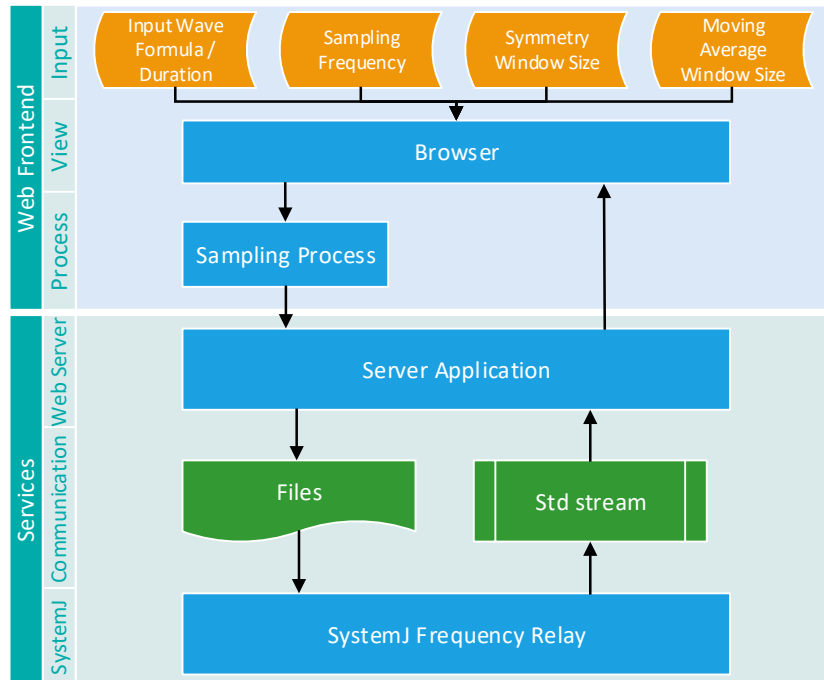


Figure 6.3 The architecture of the SystemJ frequency relay simulator

The top-level SystemJ CD of the frequency relay is shown in Figure 6.4. The CD forks two parallel reactions where the first reaction from lines 5-15 obtains values for the moving average window (`symsize`) and the symmetry window (`avesize`) via the Java properties and passes them as arguments to the main reaction of the frequency relay, which is called `DataProcessing`, at line 14. The input and the output signals of the CD `SampleIn` and `SampleCount` are also given as arguments to `DataProcessing` reaction. The second reaction at lines 17-23 terminates the program by executing the exit statement when signal `SampleIn` is not present for more than 1 second. For simulation purpose, the CD needs to provide some useful data to the server application such as results of each stage in the data processing algorithm as shown

```

1   Freq(
2       input float signal SampleIn;
3       output int signal SampleCount;->
4       trap(T){
5           {
6               int symsize, avesize;
7               try{
8                   symsize = Integer.parseInt(System.getProperty("symsize"));
9                   avesize = Integer.parseInt(System.getProperty("avesize"));
10              } catch(java.lang.Exception e) {
11                  symsize = 320;
12                  avesize = 5;
13              }
14              DataProcessing(symsize, avesize : SampleIn, SampleCount);
15          }
16          ||
17          {
18              loop {
19                  abort(SampleIn){
20                      {halt;} || {waitl(1s); exit(T);}
21                  }
22              }
23          }
24      }
25      System.out.println("{\"status\": 1 }");
26  }

```

Figure 6.4 The top-level SystemJ CD for the frequency relay.

in Figure 6.2. Furthermore, the server application needs to know when the simulation is terminated to return the result back to the user's web browser. To ease this interaction, we used standard streams as shown at line 25 where the messages are encoded in JSON format. As a result, we did not have to introduce additional output signals for only simulation purposes.

```

1      DataProcessing(int SYM_SIZE, int AVE_SIZE : input float signal SampleIn, output
2      int signal SampleCount){
3
4      float signal SymResult, AveResult;
5      { /* Moving average filter */ }
6      ||
7      { /* Correlation calculator */ }
8      ||
9      { /* Peak detector */ }
10     }
11
12                                     (a)
11     int AveIndex = -1;
12     final int AVEWS = AVE_SIZE;
13     float AveWindow[] = new float[AVEWS];
14     loop {
15         await (SampleIn);
16         AveIndex = (AveIndex + 1) % AVEWS;
17         AveWindow[AveIndex] = #SampleIn;
18         float sum = 0;
19         for (int j=0; j<AVEWS; j++){
20             sum = sum + AveWindow[(AveIndex + j) % AVEWS];
21         }
22         emit AveResult(sum / AVEWS);
23         System.out.println("{\"ave_result\": " + (sum / AVEWS) + "}");
24     }
25
26                                     (b)
25     final int L = 10;
26     int HalfSymWs = SYM_SIZE / 2;
27     int SymIndex = -1;
28     int SymWs = HalfSymWs * 2 + 1;
29     float SymWindow[] = new float[SymWs];
30     loop {
31         await (AveResult);
32         SymIndex = (SymIndex + 1) % SymWs;
33         SymWindow[SymIndex] = #AveResult;
34         int CorIndex = (SymIndex - HalfSymWs + SymWs) % SymWs;
35         float CorResult = 0;
36         for (int k=0; k<=HalfSymWs; k++) {
37             float x1 = SymWindow[(CorIndex + k) % SymWs];
38             float x2 = SymWindow[(CorIndex - k + SymWs) % SymWs];
39             CorResult = CorResult + (L + x1) * (L + x2);
40         }
41         emit SymResult(CorResult);
42         System.out.println("{\"sym_val\": " + CorResult + "}");
43     }
44
45                                     (c)
44     float first = 10000f, second = 10000f, third = 10000f;
45     boolean FirstPeak = true;
46     int SampleCountVar = 0;
47     int SkipSamples = 0;
48     loop {
49         await (SymResult);
50         if (SkipSamples == 0) {
51             SampleCountVar++;
52             third = second;
53             second = first;
54             first = #SymResult;
55             if (second > first && second > third){
56                 if (FirstPeak) {
57                     FirstPeak = false;
58                     SampleCountVar = 1;
59                 } else {
60                     int tyi = SampleCountVar - 1;
61                     SampleCountVar = 1;
62                     System.out.println("{\"sample_count\": " + tyi + "}");
63                     emit SampleCount("{\"sample_count\": " + tyi + "}");
64                 }
65             }
66         } else {
67             SkipSamples--;
68         }
69     }
70
71                                     (d)

```

Figure 6.5 DataProcessing reaction implementation.

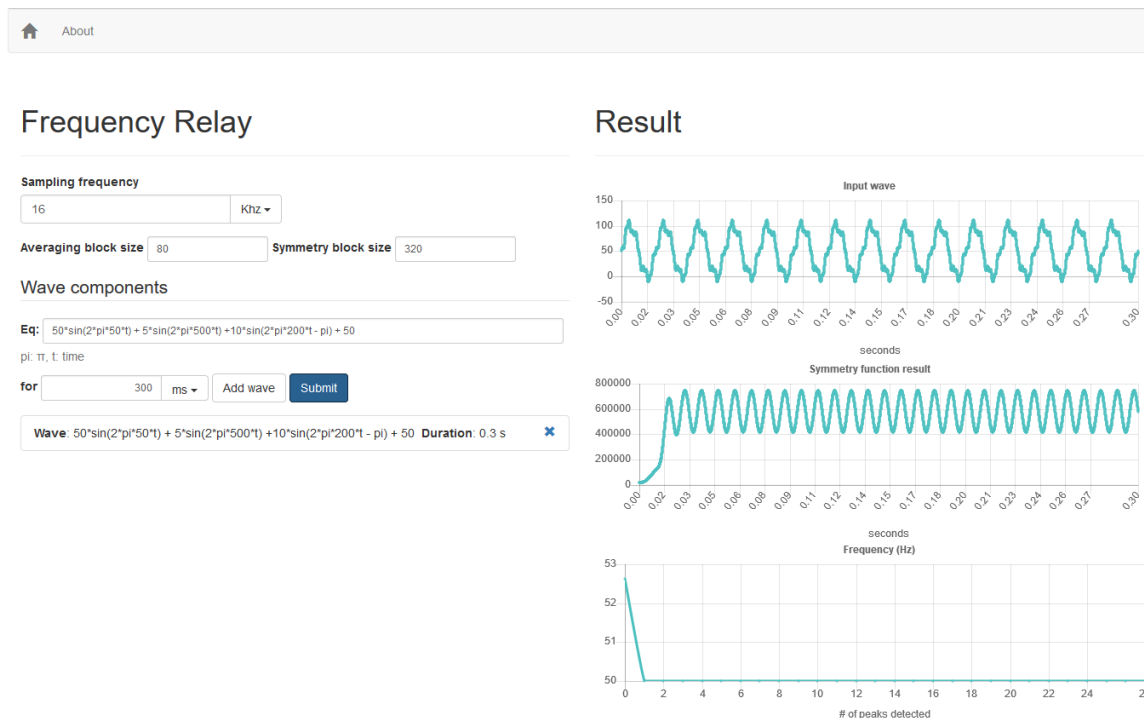


Figure 6.6 Frequency relay web front-end

Figure 6.5 shows implementation of the DataProcessing reaction, which is data computation part of the frequency relay. The reaction further consists of three children reactions for the moving averaging filter, the correlation calculator, and the peak detector blocks, interacting with each other via signals SymResult and AveResult as shown at lines 4-9 in Figure 6.5(a). The moving average filter shown in Figure 6.5(b) creates an array of a size equal to the average window size (AVE_SIZE), and calculates the average number for the current index in the array every time when the input sample SampleIn is captured. The resultant number is then passed to the correlation calculator via signal AveResult (line 22). The correlation calculator (Figure 6.5(c)) implements Equation 1 that calculates correlation within the range specified by the symmetry window size (SYM_SIZE) and delivers the result to the peak detector block via signal SymResult (line 41). The peak detector emits signal SampleCount when it detects a peak value from the results of the correlation calculator.

The SystemJ frequency relay delivers the intermediary results and the output of the peak detector unit to the server application, which further sends the results back to the user's web browser via an HTTP response. The layout of the web front-end is shown in Figure 6.6.

6.2 Industrial Automation and Robotics Collaborative System

Industrial automation consists of computers or robots, machines and control systems for handling manufacturing processes without significant human intervention. In this section, we demonstrate designing of software system using SystemJ for controlling filling and bottling processes for making personalised drinks. The machine which the SystemJ program controls is called a bottling station. It has several submodules that provides automated filling of the liquid and capping of bottlings, and a lid loader that can be used to provide lids that will be used for capping the bottles. Empty bottles as well as final products are handled by a Baxter robot, which can pick up and place bottles at certain positions.

The fully automated system including the Baxter robot is shown in Figure 6.8. There are three major functional parts: Bottling station, Lid loader and Baxter robot. The machines are equipped by a mixture of microcontrollers – e.g. ARM processors or custom-designed processors within FPGA-based SoC. These computers are connected to a local network implemented by a local router that supports wired (Ethernet) and Wi-Fi connections. The bottling machine consists of four constituent machines, the conveyor, rotary turntable, filler and capper. Baxter left arm (LA) and Baxter right arm (RA) are used to move lids and bottles to their destination positions on the bottling station, respectively. The physical arrangement and positions of the three machines are fixed. Operation of the system is supported by a number of photo-eyes that detect the passage of bottles at each point of the bottling station submodule. The presence of a bottle in front of a photo-eye is used for synchronisation of operations of the bottling process. Baxter robot, bottling machine and lid loader work together for completion of a manufacturing cycle. During the cycle, bottles are made available at the bottle load point.

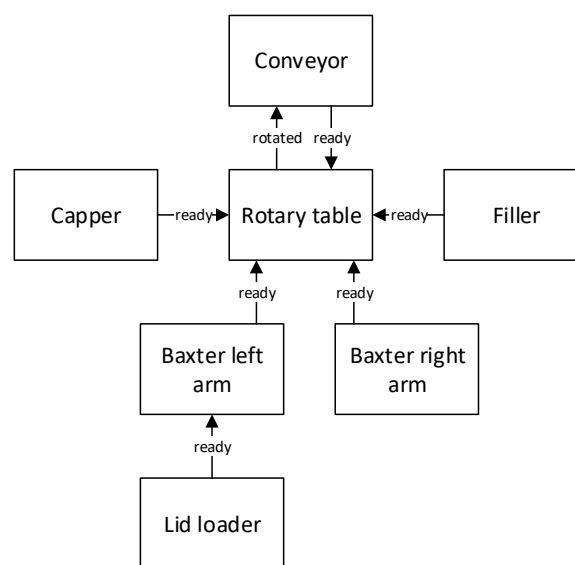


Figure 6.7 CDs for the bottling station, a Baxter robot and a lid loader

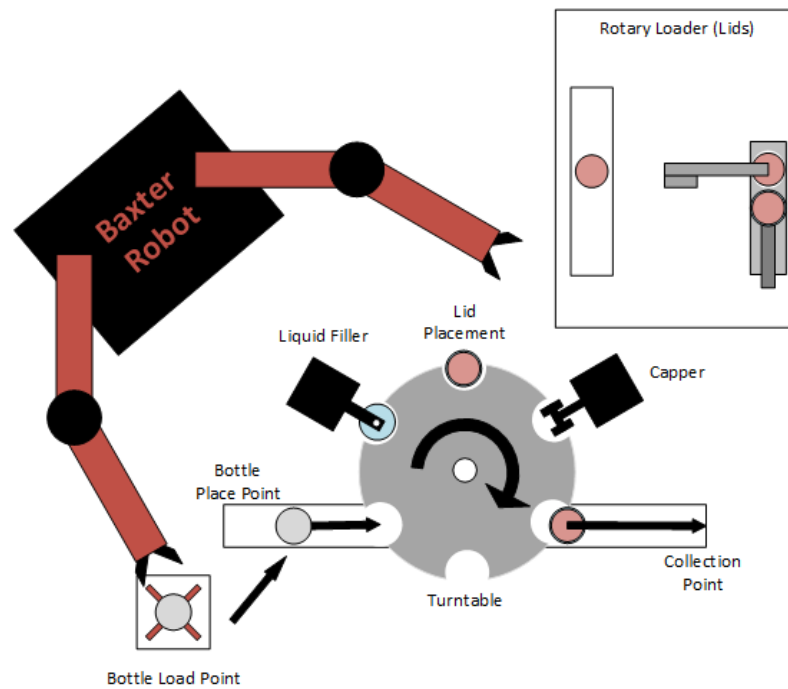


Figure 6.8 Simplified overview of the Bottling Station

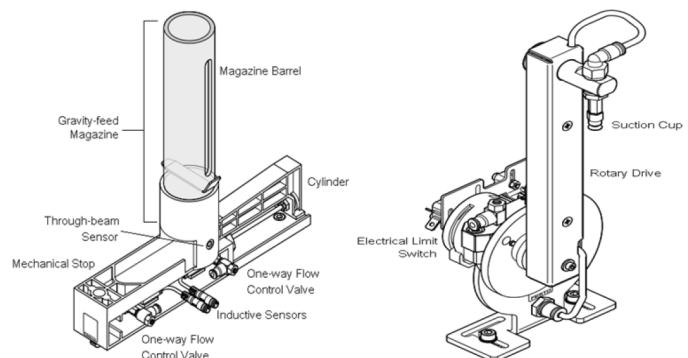
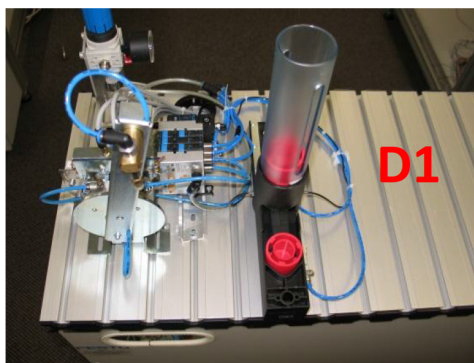


Figure 6.9 The lid loader station

The Baxter robot then picks up an empty bottle and places it on the conveyor at the bottle place point (left end of conveyor). The bottle travels along the conveyor until it reaches the turntable and the presence of the bottle at that point is recognised by a photo-eye. The turntable rotates and moves the bottle in steps of 60 degrees; the first step moves the bottle until it is below the liquid filler. The bottle is filled, and the turntable rotates once more, until it is at the lid placement point. Baxter picks up a lid that has been dispensed from the lid loader (which supplies lids from a magazine to the pick-up point) and places it on the bottle. The turntable rotates one more step, where the capper then pushes and screws the lid onto the bottle. Finally, the turntable rotates one more step, where the finished bottle is taken over by the conveyor and moved to the collection point at the right end of the conveyor.

An overview of software architecture for the overall system is shown in Figure 6.7. The system consists of 7 CDs in total where each CD controls a part of the overall bottling process via I/O signals (not shown in the figure). Rotary table CD manages global state of the bottling

machine because the table should only rotate when all other CDs are done with processing bottles, and this information is delivered to the rotary table CD via channels called `ready`. Additionally, the Conveyor CD is responsible to check if there are no more bottles remaining in the station, which in that case prevents the table to rotate by sending a ‘busy’ message to the Rotary table CD. In order to keep track of a number of bottles in the station, the rotary table CD notifies the conveyor CD via a channel `rotated` whenever the table has been rotated. Unlike other CDs, the lid loader CD does not send a ready message to the rotary table CD, instead it sends a message to Baxter left arm CD so that a lid can be picked up by the Baxter once it is ready. In the following subsections, we are going to illustrate operations for each CD in Figure 6.7 using concrete SystemJ implementations.

6.2.1 Lid loader CD

A pictorial illustration of the lid loader is shown in Figure 6.9 which consists of three major mechatronic components.

1. A *magazine barrel*, which is independently loaded (manually or by another manipulator) and can hold up to 5 lids. The magazine has a photo-eye sensor that can generate an indication when empty.
2. An *ejecting cylinder (pusher)*, which needs to retract to its initial state in order to be able to push a lid out of the magazine to an intermediate position from where the lid becomes available to be taken by the rotary drive arm. The pusher generates feedback signals for the controller when it reaches stable positions (fully retracted or fully extended).
3. A *rotary drive arm* is equipped with a suction cup that enables transfer of a lid from the position where it was left by the pusher to a pick up (destination) location. The arm has two stable positions/states that indicate (1) the arm is at the source position (retracted), where it finds a lid for transfer, and (2) the arm is at the destination position (extended) where the lid is handed over to another machine. The arm provides signals that indicate its current position. Also, added to the top of the arm is a suction cup which allows grabbing a lid at the source location and releasing it at the destination location.

This lid loader can be controlled via sets of actuators and sensors which are accessed and interfaced with the lid loader CD via the input and the output signals as listed below.

- `pusherExtend`: Continuous emission of this signal causes the pusher to extend. Absence of this signal will cause the pusher to retract.
- `vacOn`: Continuous emission of this signal will turn the vacuum on.
- `armSource`: Continuous emission of this signal causes the arm swivels to the source position to pick up a lid.
- `armDest`: Continuous emission of this signal causes the arm swivels to the destination position to drop off a lid.
- `pusherRetracted`: Indicates the pusher has been retracted.
- `pusherExtended`: Indicates the pusher has been extended.
- `WPgripped`: Indicates the lid has been picked up by the suction cup.
- `armAtSource`: indicates the rotary drive is reached to the source to pick up a workpiece
- `armAtDest`: indicates the rotary drive is reached to the destination
- `empty`: indicates the magazine barrel is empty


```

1  LidLoader(
2      input signal pusherRetracted, pusherExtended,
3          WPgripped, armAtSource, armAtDest, empty, request;
4      output signal pusherExtend, vacOn, armSource, armDest; )->{
5
6      while(true) {
7          await(!empty);
8          await(request);
9          abort(armAtDest) { sustain armDest; }
10         abort(pusherExtended) { sustain pusherExtend; }
11         await(pusherRetracted);
12         abort(armAtSource) { sustain armSource; }
13         trap(T){
14             { sustain vacOn; }
15             ||
16             {
17                 await(WPgripped);
18                 abort(armAtDest) { sustain armDest; }
19                 exit(T);
20             }
21         }
22         await(!WPgripped);
23         abort(armAtSource){ sustain armSource; }
24     }
25 }

```

Figure 6.11 SystemJ source code for the lid loader CD

A concrete implementation of the lid loader controller CD is shown in Figure 6.11. The CD first checks if the magazine barrel is empty via signal `empty` at line 7, and waits until there is at least one lid in the barrel. Next, it waits until request for a lid is placed by the Baxter robot or a human operator via signal `request`. A sequence of control operations is followed next. The lid loader CD first emits `armDest` until the rotary arm swivels back to the destination point so that it will not block a lid that is being pushed. The pusher is then extended fully so that the lid is placed to the pick-up location. The CD waits until the pusher is retracted back to the original location and swivels the arm to pick-up the lid. These operations are shown at lines 9-12. The next operation is to place the lid on the destination point so that Baxter can pick up the lid. To achieve this, vacuum on the tip of the rotating arm is turned on by continuously emitting signal `vacOn` using the `sustain` statement (line 14). While the signal is being emitted, a second reaction, spawned in lines 16-20, moves the arm to the destination point, which is shown at line 17-18. Presence of signal `WPgripped` indicates the lid is tightly ticked to the

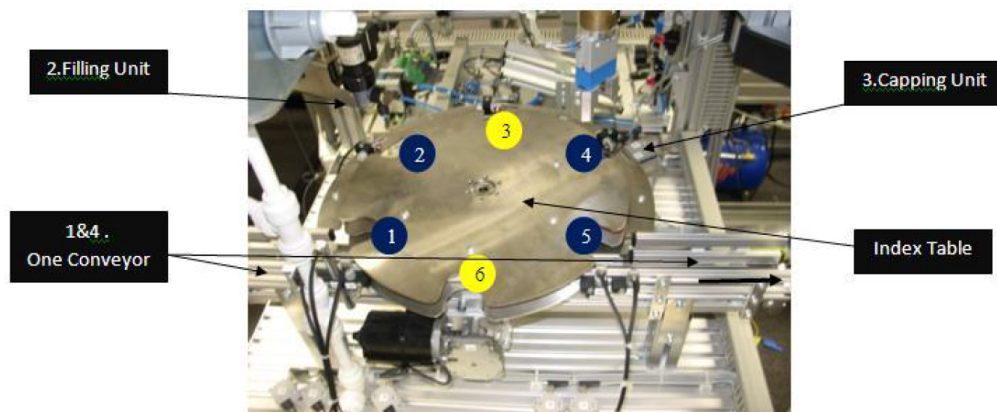


Figure 6.10 Bottling machine main components

suction cup. When the arm reaches to the destination point, it stops emitting `vacOn` by interrupting the reactions via `trap` statement (line 19) and swivels the arm to the source position.

6.2.2 Rotary table CD

As explained previously, the rotary table CD receives status of rest of CDs for the bottling station. This information is used to rotate a table that processes bottles in a sequence of operations. The following input and output signals are present for the rotary table CD.

- `tableAlignedWithSensor`: Present when the index positions of the table are aligned with photo-eye sensors.
- `bottleAtPos5`: Present when the bottle is at position 5.
- `rotaryTableTrigger`: Turns the rotary table while this signal is present.

As shown in Figure 6.10, the rotary table has 6 positions (position 3 and 6 are unused). In order to shift bottles to the next station (index), each time the table makes 60 degree rotation. Figure 6.13 shows SystemJ source code for the rotary table CD. The rotary table CD waits until all other controllers complete their operation and notify the rotary CD via the `ready` signals (line 11). When all of the controllers are ready, the rotary CD finally checks if there is still a bottle at position 5 and waits until the bottle is cleared via absence of signal `bottleAtPos5` (lines 13-15). Rotation of the table is controlled by presence of signal `rotaryTableTrigger`, which enables a motor when the signal is present. The controller sustains this signal for 100ms, as shown at lines 17-19, and waits until the table indexes are aligned with photo-eye sensors, which is shown at lines 21-22. The controller waits for another 500ms before it reiterates the same operation.

1	<code>reaction checkReady(: input Boolean channel ReadyIn, output signal ReadyOut)</code>
2	<code>{</code>
3	<code> loop{</code>
4	<code> trap(T){</code>
5	<code> {receive ReadyIn; pause; pause; exit(T);}</code>
6	<code> </code>
7	<code> { if(#ReadyIn){ sustain ReadyOut; } }</code>
8	<code> }</code>
9	<code> pause;</code>
10	<code> }</code>
11	<code>}</code>
12	
13	

Figure 6.12 checkReady reaction

Receiving ready statuses from the controllers is done via blocking channels. To prevent the main control loop being blocked due to the channel communication, these operations are isolated in separate concurrent reactions and the statuses are converted into non-blocking signals as shown at lines 27-35. Status checking logic is implemented in a reaction called `checkReady`, which has two arguments: an input channel and an output signal. The role of this reaction is to sustain the output signal when the received channel value is true. The implementation of the reaction `checkReady` is shown in Figure 6.12.

```

1  RotaryCD(
2      input signal tableAlignedWithSensor, bottleAtPos5;
3      output signal rotaryTableTrigger;
4      input Boolean channel fillerReady, capperReady, convReady, baxterReady;
5      output Boolean channel rotated;
6  )->{
7      signal fR, cR, cvR, bR;
8
9      {
10         loop{
11             {await(fR);}||{await(cR);}||{await(cvR);}||{await(bR);}
12
13             present(bottleAtPos5) {
14                 await(!bottleAtPos5);
15             }
16
17             trap(T){
18                 {sustain rotaryTableTrigger;}||{waitl(100 ms); exit(T);}
19             }
20
21             await(tableAlignedWithSensor);
22             send rotated(SBoolean.TRUE);
23             waitl(500 ms);
24         }
25     }
26     ||
27     {
28         checkReady(:fillerReady, fR)
29         ||
30         checkReady(:capperReady, cR)
31         ||
32         checkReady(:convReady, cvR)
33         ||
34         checkReady(:baxterReady, bR)
35     }
36 }

```

Figure 6.13 SystemJ source code for the Rotary table CD

6.2.1 Capper CD

The capper CD is responsible for sealing the final product with a plastic cap before the item exits the bottling machine station. The CD consists of the following input and output signals:

- bottleAtPos4 – Present when a bottle is at position 4
- gripperZAxisLowered – Present when the gripper/capper unit is fully lowered
- gripperZAxisLifted – Present when the gripper/capper unit is fully lifted
- gripperTurnHomePos – Present when the gripper is at the initial position
- gripperTurnFinalPos – Present when the gripper is fully turned
- cylPos5ZaxisExtend – brings the gripper down (absence of this signal will bring the gripper up)
- gripperTurnRetract – untwists the gripper
- gripperTurnExtend – twists the gripper
- capGripperPos5Extend – Grips the cap (absence of this signal will release the cap)
- cylClampBottleExtend – Clamps the bottle (absence of this signal will unclamp the bottle)

Upon initialization, the capper CD sends its status “true” (line 8) to the Rotary CD, indicating that there is no bottle at position 4. When a bottle is detected (line 11), the CD sends this update (status “false”) to the Rotary CD via a channel and checks if the gripper is fully lifted before stating the capping operation (lines 12-13). A sequence of capping operation done next via a set of input and output signals (lines 14-31). The reactions spawned at lines 16-29 show parallel operations where the capper tightens the cap while the gripper is pushed down.

```
1  CapperCD(  
2      input signal bottleAtPos4, gripperZAxisLowered,gripperZAxisLifted,  
3          gripperTurnHomePos,gripperTurnFinalPos;  
4      output signal cylPos5ZaxisExtend,gripperTurnRetract,gripperTurnExtend,  
5          capGripperPos5Extend,cylClampBottleExtend;  
6      output SBoolean channel capperReady;)-> {  
7  
8      send capperReady(SBoolean.TRUE);  
9      {  
10         loop{  
11             await(bottleAtPos4);  
12             send capperReady(SBoolean.FALSE);  
13             abort(gripperZAxisLifted){halt;}  
14             abort(gripperTurnHomePos){sustain gripperTurnRetract;}  
15             trap(T){  
16                 {sustain cylPos5ZaxisExtend;}  
17                 ||  
18                 {  
19                     await(gripperZAxisLowered);  
20  
21                     {sustain capGripperPos5Extend;}  
22                     ||  
23                     {sustain gripperTurnExtend;}  
24                     ||  
25                     {  
26                         await(gripperTurnFinalPos);  
27                         exit(T);  
28                     }  
29                 }  
30             }  
31             await(gripperZAxisLifted);  
32             send capperReady(SBoolean.TRUE);  
33             await(!bottleAtPos4);  
34         }  
35     }  
36 }
```

Figure 6.14 SystemJ source code for the capper CD

The CD then updates its status back to “true” (line 32) and waits until the bottle is moved to the next position (line 33).

6.2.2 Conveyor CD

This CD controls the movement of conveyor belt that is an entry point to the bottling station, e.g. position 1. In addition, this CD also detects presence of a bottle at position 1 and this information is transferred to the Rotary CD via the channel `convReady`. This CD interacts

```

1  ConveyorCD(
2      input signal bottleAtPos1, bottleLeftPos5;
3      output signal motConveyorOnOff;
4      input Boolean channel rotated;
5      output Boolean channel convReady, incomingBaxter;
6  )
7  ->
8  {
9      signal rotaryRotated, empty, insert;
10     {
11         loop{
12             trap(T){
13                 {
14                     await(bottleAtPos1);
15                     pause;
16                     exit(T);
17                 }
18                 ||
19                 {
20                     await(immediate empty);
21                     pause;
22                     send convReady(false);
23                 }
24             }
25             pause;
26             emit insert;
27             send convReady(false);
28             pause;
29
30             {send convReady(true);} || {await (rotaryRotated);}
31
32             pause;
33         }
34     }
35     ||
36     RotaryQueue(:insert, rotaryRotated, empty, incomingBaxter)
37     ||
38     {
39         loop{
40             receive rotated;
41             emit rotaryRotated;
42             pause;
43         }
44     }
45     ||
46     {sustain motConveyorOnOff;}
47 }

```

Figure 6.15 SystemJ source code for the Conveyor CD

with the sensors and actuators via the following input and output signals.

- `bottleAtPos1` – Present when a bottle is arrived at position 1 of the index table.
- `bottleLeftPos5` – Present when there is a bottle, which left from position 5 of the index table, still on the conveyor
- `motConveyorOnOff` – Enabling/disabling the conveyor motor

The Conveyor CD first checks the presence of a bottle at position 1 (line 14). When the bottle is detected, the preemptive `exit` is executed at line 16 and the reaction continues execution from line 25. The Conveyor belt is also responsible to track the total number of bottles being processed in the bottling station. This is done in a reaction called `RotaryQueue`, which implements a circular buffer whose index indicates a presence of a bottle at specific position. When there is no bottle in the station, this information is delivered to the Conveyor belt via signal `empty`, whose status is also transferred to the Rotary CD via the channel `convReady` (at lines 20-22). For example, when there no bottle is currently being processed, the rotary table will not turn since the status “false” will be sent to the Rotary CD via `convReady` at line 27. Once the rotary table is rotated for 60 degrees, the Rotary table CD sends this information to the Conveyor CD via channel `rotated` (line 40), which is broadcasted locally within the Conveyor CD via signal `rotaryRotated`. This signal triggers the Conveyor CD to start capturing the next incoming bottle at position 1 (line 30).

6.2.3 Filler CD

The Filler CD is responsible to fill an empty bottle with liquid. This CD consists of the following input and output signals.

- `bottleAtPos2` – Present when the bottle is at position 2
- `dosUnitEvac` – Present when a pressure canister is at bottom
- `dosUnitFilled` – Present when a pressure canister is at top

```

1  FillerCD(
2      input signal bottleAtPos2,dosUnitEvac,dosUnitFilled;
3      output signal valveInjectorOnOff, valveInletOnOff, dosUnitValveRetract,
4      dosUnitValveExtend ;
5      output Boolean channel fillerReady;->{
6
7          send fillerReady(Boolean.TRUE);
8
9          {
10             loop{
11                 await(bottleAtPos2);
12
13                 send fillerReady(Boolean.FALSE);
14
15                 abort(dosUnitFilled){
16                     {
17                         sustain valveInjectorOnOff;
18                     }
19                     ||
20                     {
21                         sustain dosUnitValveRetract;
22                     }
23                 }
24
25                 abort(dosUnitEvac){
26                     {
27                         sustain valveInletOnOff;
28                     }
29                     ||
30                     {
31                         sustain dosUnitValveExtend;
32                     }
33                 }
34
35                 send fillerReady(Boolean.TRUE);
36             }
37         }
38     }

```

Figure 6.16 SystemJ source code for the Filler CD

- valveInjectorOnOff – Turns on or off the valve injector (absence of this signal will turn off the injector)
- valveInletOnOff – Opens the inlet valve (absence of this signal will close the inlet)
- dosUnitValveRetract – brings the pressure canister to top
- dosUnitValveExtend – brings the pressure canister to bottom

Similar to the other CDs, the Filler CD sends its initial status to the Rotary CD (line 7) and checks for the presence of a bottle via signal (**bottleAtPos2** at line 12). The CD then carries out the filling operation which is shown at lines 15-33.

Chapter 7 Conclusion

Increase in complexity of computer systems brought in various design tools and techniques including formal programming languages, tools, and correct-by-construction design methodologies. This book introduced a system-level design language called SystemJ, which is equipped with formal Globally Asynchronous Locally Synchronous (GALS) model of computation (MoC). A SystemJ program consists of a set of asynchronous processes called clock-domains that are comprised of closely related concurrent activities called reactions. The clock-domains run asynchronously with each other at the top-level and this creates globally asynchronous execution model. The book introduced both synchronous and asynchronous execution semantics of SystemJ language and how they are captured in a program using kernel and derived SystemJ statements. SystemJ defines well established translation rules that translate a SystemJ program into an intermediate representation called Asynchronous GRaph code (AGRC). AGRC is a foundation for both program analysis and efficient back-end code generation for SystemJ programs, which can be deployed to any JVM-enabled execution platforms. This book introduced two main case-studies where the SystemJ language can be used in designing software systems.

As for future work, we are planning to enhance the language's capability in processing data algorithms by introducing so called "data modules", which can be modularised and implemented using various software/hardware technologies. We foresee this will open new research opportunities including target platforms, applications, and analysis techniques.

Appendix 1 Lists of the Elements and Attributes of the Configuration File

The following tables provide a short description of each element used in a configuration file.

Element	Description	Nested Elements
System	A top-level entity describing a SystemJ system. Every configuration file should have a single System as a root element.	Interconnection*, SubSystem+
Interconnection	Interconnection is a collection of Link	Link*
Link	Link is a collection of Interface that provides a way to exchange SystemJ channel data between subsystems running on remote machines.	Interface+
Interface	Provides a physical location of subsystems.	-
SubSystem	Consists of ClockDomains	ClockDomain+, Scheduler*
ClockDomain	SystemJ clock-domain	iSignal*, iChannel*, oChannel*
iSignal	Input signal	
oSignal	Output signal	-
iChannel	Input channel	-
oChannel	Output channel	-
Scheduler	Schedules clock-domain execution	ClockDomain*

* (zero or more of this element), + (one or more of this element)

List of Attributes

- System

None

- Interconnection

None

- Link

Attribute	Description	Required
Type	Specifies a type of link. Use Destination if the remote machine is addressable using a global identifier (e.g. IP address). Use Local if the remote machine is connected via a port such as COM1.	Yes

- Interface

Attribute	Description	Required
SubSystem	Name of a subsystem connected via a link.	Yes
Class	Fully qualified Java class name that implements this interface	Yes
Args	An argument passed to the class file specified in Class .	No

- SubSystem

Attribute	Description	Required
Name	Name of this subsystem.	Yes
Local	If true, load and execute all clock-domains that belong to this subsystem.	No

- ClockDomain

Attribute	Description	Required
Name	Name of this clock-domain.	Yes
Class	Fully qualified Java class name of this clock-domain.	Yes

- **iSignal**

Attribute	Description	Required
Name	Name of the signal.	Yes
Class	Fully qualified Java class name that captures inputs from environment.	Yes
IP	(If TCPReceiver is used for Class) Local IP address that the server will bind to.	(Yes for TCPReceiver)
Port	(If TCPReceiver is used for Class) Port number of the server.	(Yes for TCPReceiver)

- **oSignal**

Attribute	Description	Required
Name	Name of the signal.	Yes
Class	Fully qualified Java class name that provides an output to environment.	Yes
IP	(If TCPSender is used for Class) IP address of the remote machine.	(Yes for TCPSender)
Port	(If TCPSender is used for Class) Connects to this port number on the remote machine.	(Yes for TCPSender)

- **iChannel**

Attribute	Description	Required
Name	Name of the input channel.	Yes
To	Name of the output channel that this input channel is connected to. The format is <ClockDomainName>.<ChannelName>, e.g. CD1.CH.	Yes

- **oChannel**

Attribute	Description	Required
Name	Name of the output channel.	Yes
From	Name of the input channel that this output channel is connected to. The format is <ClockDomainName>.<ChannelName>, e.g. CD2.CH.	Yes

- **Scheduler**

Attribute	Description	Required
Class	Fully qualified Java class name that schedules clock-domains. (e.g. <code>systemj.scheduler.ThreadScheduler</code>)	Yes
Args	An argument passed to the class file specified in Class.	No

The schema file of the SystemJ configuration file can be found at <https://github.com/hjparker/gcf-schema>.

Appendix 2 SystemJ Tools and Example Programs

The SystemJ compiler can be downloaded from <https://github.com/hjparker/systemj-release> whose usage is explained in Chapter 5. A set of example programs are also available at <https://github.com/hjparker/systemj-demo>. Refer to the README file in the github example repository on how to build and run the programs.

References

- [1] R. Roestenburg, R. Bakker, and R. Williams, *Akka in action*. Manning Publications Co., 2015.
- [2] G. Berry and G. Gonthier, “The Esterel synchronous programming language: Design, semantics, implementation,” *Sci. Comput. Program.*, vol. 19, no. 2, pp. 87–152, 1992.
- [3] A. Malik, Z. Salcic, P. S. Roop, and A. Girault, “SystemJ: A GALS language for system level design,” *Comput. Lang. Syst. Struct.*, vol. 36, no. 4, pp. 317–344, 2010.
- [4] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud, “The synchronous data flow programming language LUSTRE,” *Proc. IEEE*, vol. 79, no. 9, pp. 1305–1320, 1991.
- [5] P. LeGuernic, T. Gautier, M. Le Borgne, and C. Le Maire, “Programming real-time applications with SIGNAL,” *Proc. IEEE*, vol. 79, no. 9, pp. 1321–1336, 1991.
- [6] C. A. R. Hoare, “Communicating sequential processes,” in *The origin of concurrent programming*, Springer, 1978, pp. 413–443.
- [7] K. Van Berkel and A. Bink, “Single-track handshake signaling with application to micropipelines and handshake circuits,” in *Advanced Research in Asynchronous Circuits and Systems, 1996. Proceedings., Second International Symposium on*, 1996, pp. 122–133.
- [8] E. A. Lee, “The problem with threads,” *Computer*, vol. 39, no. 5, pp. 33–42, 2006.
- [9] R. Sinha, V. Vyatkin, Z. Salcic, and H. J. Park, “Competitors or Cousins? Studying the parallels between distributed programming languages SystemJ and IEC61499,” in *Emerging Technology and Factory Automation (ETFA), 2014 IEEE*, 2014, pp. 1–7.
- [10] N. A. Lynch, *Distributed algorithms*. Morgan Kaufmann, 1996.
- [11] Z. Salcic, M. Nadeem, H. Park, and J. Teich, “Optimizing Latencies and Customizing NoC of Time-Predictable Heterogeneous Multi-core Processor,” in *Embedded Multicore/Many-core Systems-on-Chip (MCSoc), 2016 IEEE 10th International Symposium on*, 2016, pp. 233–240.
- [12] Z. Salcic, H. Park, J. Teich, A. Malik, and M. Nadeem, “NoC-HMP: A Heterogeneous Multicore Processor for Embedded Systems Designed in SystemJ,” *ACM Trans. Des. Autom. Embed. Syst.*
- [13] Z. Li *et al.*, “Using design space exploration for finding schedules with guaranteed reaction times of synchronous programs on multi-core architecture,” *J. Syst. Archit.*, p. , 2016.
- [14] M. Schoeberl, “Jop: A java optimized processor for embedded real-time systems,” Technical University of Denmark/Danmarks Tekniske Universitet, Department of Informatics and Mathematical Modeling/Institut for Informatik og Matematisk Modellering, 2008.
- [15] M. Nadeem, M. Biglari-Abhari, and Z. Salcic, “RJOP-A customized Java processor for reactive embedded systems,” in *Design Automation Conference (DAC), 2011 48th ACM/EDAC/IEEE*, 2011, pp. 1038–1043.
- [16] M. Nadeem, M. Biglari-Abhari, and Z. Salcic, “GALS-JOP: A Java embedded processor for GALS reactive programs,” in *Dependable, Autonomic and Secure Computing (DASC), 2011 IEEE Ninth International Conference on*, 2011, pp. 292–299.
- [17] M. Nadeem, M. Biglari-Abhari, and Z. Salcic, “JOP-plus-A processor for efficient execution of java programs extended with GALS concurrency,” in *Design Automation Conference (ASP-DAC), 2012 17th Asia and South Pacific*, 2012, pp. 17–22.
- [18] M. Nadeem, H. Park, Z. Li, M. Biglari-Abhari, and Z. Salcic, “GALS-CMP: chip-multiprocessor for GALS embedded systems,” in *International Conference on Architecture of Computing Systems*, 2013, pp. 147–158.
- [19] H. Park, A. Malik, and Z. Salcic, “Scheduling globally asynchronous locally synchronous programs for guaranteed response times,” *ACM Trans. Des. Autom. Electron. Syst. TODAES*, vol. 20, no. 3, p. 40, 2015.
- [20] H. Park, Z. Li, A. Malik, and Z. Salcic, “Times square—marriage of real-time and logical-time in GALS and synchronous languages,” *J. Signal Process. Syst.*, vol. 84, no. 1, pp. 163–180, 2016.
- [21] H. Park, A. Malik, and Z. Salcic, “Compiling and verifying SC-SystemJ programs for safety-critical reactive systems,” *Comput. Lang. Syst. Struct.*, vol. 44, pp. 251–282, 2015.